Search and Optimization

Professor Nihad Karim Chowdhury
Department of Computer Science and Engineering
University of Chittagong

Lecture Overview

- **Problem Solving via Search:** Uninformed search algorithms (BFS, DFS).
- **Heuristic Search:** Informed search algorithms (A*, greedy search).
- **Optimization Problems:** Hill climbing, Simulated annealing, Genetic algorithms.

Problem Solving via Search

- **What is Search?**
  - Search in AI is the process of exploring paths to find a solution in a state space.
  - State Space Representation: A graph where nodes represent states and edges represent transitions.
  - Objective: Find a path from the initial state to a goal state.
- **Search Algorithms Classification:**
  - Uninformed (Blind) Search: No additional information about goal location (BFS, DFS).
  - Informed Search: Uses heuristics to guide the search (A*, Greedy Best-First Search).

Uninformed Search

- **Uninformed Search**
  - **Definition:** Algorithms that explore the search space without additional knowledge of goal proximity.
  - **Types:** BFS (Breadth-First Search), DFS (Depth-First Search).

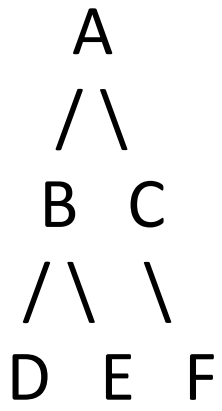
Breadth-First Search (BFS)

Why is it called "Breadth-First Search"?

- **Breadth** refers to **exploring all nodes at the current level (or depth)** before moving on to the next level. In BFS, the algorithm starts at a given node (usually the root or the starting point) and explores all its neighboring nodes before moving on to the neighbors of those neighbors.
- The word **first** refers to the fact that nodes are explored in order of their distance from the starting node. BFS explores the search space in increasing order of distance from the root or starting node.
- **Breadth-first** literally means that the algorithm prioritizes the "breadth" or **width of the search** by exploring each layer or level of the search space before diving deeper.

Breadth-First Search (BFS)

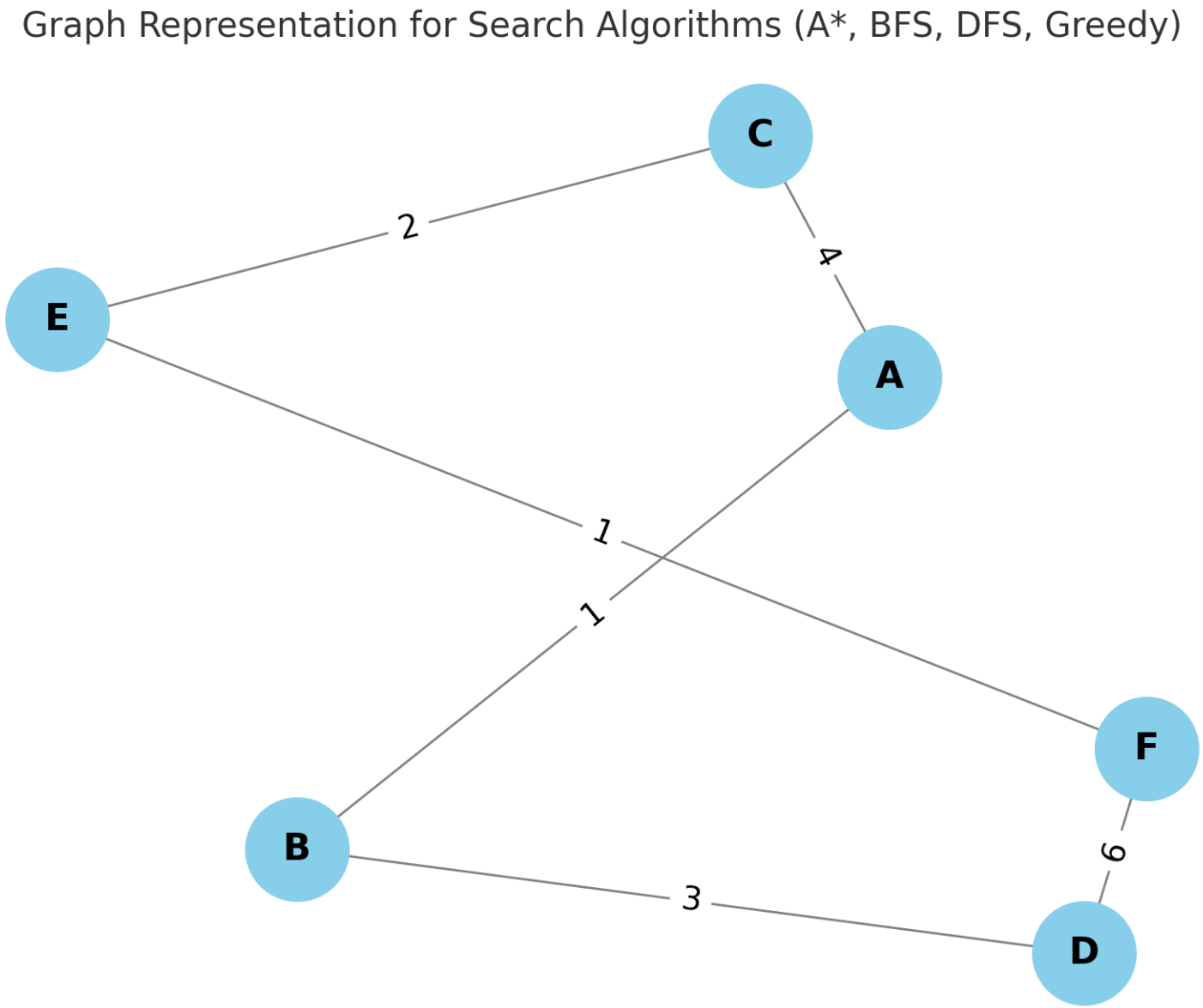
Visualization Example (Tree)



BFS explores the tree level by level, meaning:

- First explores node A.
- Then, nodes B and C (neighbors of A).
- Then, nodes D, E, and F (neighbors of B and C).

Graph Representation for Search Algorithms



Breadth-First Search (BFS)

Breadth-First Search (BFS) step by step using the same graph example, showing the actions, updates to the queue, explored set, and parent dictionary at each step.

Graph Setup:

Nodes: A, B, C, D, E, F

Edges (connections): A-B, A-C, B-D, C-E, D-F, E-F

Start Node: A

Goal Node: F

BFS Process:

BFS uses a queue to explore nodes level by level. It ensures that nodes are explored in the shortest path order.

Breadth-First Search (BFS)

- Step 1: Initialize
  - Queue: [A]
  - Explored: {}
  - Parent: {A: None} (Since A is the starting point, it has no parent).

Breadth-First Search (BFS)

- Step 2: Explore A
  - Dequeue A from the queue (exploring A).
  - Add A to the explored set.
  - Enqueue A's neighbors (B, C) into the queue.
    - Queue: [B, C]
    - Explored: {A}
    - Parent: A: None,
      B: A,
      C: A

Breadth-First Search (BFS)

- Step 3: Explore B
  - Dequeue B from the queue.
  - Add B to the explored set.
  - Enqueue B's neighbor (D) into the queue.
    - Queue: [C, D]
    - Explored: {A, B}
    - Parent: A: None,
      B: A,
      C: A,
      D: B

Breadth-First Search (BFS)

- Step 4: Explore C
  - Dequeue C from the queue.
  - Add C to the explored set.
  - Enqueue C's neighbor (E) into the queue.
    - Queue: [D, E]
    - Explored: {A, B, C}
    - Parent: A: None,
      B: A,
      C: A,
      D: B,
      E: C

Breadth-First Search (BFS)

- Step 5: Explore D
  - Dequeue D from the queue.
  - Add D to the explored set.
  - Enqueue D's neighbor (F) into the queue.
    - Queue: [E, F]
    - Explored: {A, B, C, D}
    - Parent: A: None,
      B: A,
      C: A,
      D: B,
      E: C
      F: D

Breadth-First Search (BFS)

- Step 6: Explore E
  - Dequeue E from the queue.
  - Add E to the explored set.
  - Enqueue: The neighbor of E are F and C, but F is already in the queue and C is explored earlier, so no new nodes are added in queue
    - Queue: [F]
    - Explored: {A, B, C, D, E}
    - Parent: A: None,
      B: A,
      C: A,
      D: B,
      E: C
      F: D

Breadth-First Search (BFS)

- Step 7: Explore F (Goal Reached)
  - Dequeue F from the queue.
  - Add F to the explored set.
  - Since F is the goal, BFS terminates.
    - Queue: [Empty]
    - Explored: {A, B, C, D, E, F}
    - Parent(no changes since we reached the goal):

      A: None,

      B: A,

      C: A,

      D: B,

      E: C

      F: D

Breadth-First Search (BFS)

- Step 7: Reconstruct the Path

Now, we reconstruct the path from the goal (F) back to the start (A) using the parent dictionary.

- Start at F:F's parent is D.
- Move to D:D's parent is B.
- Move to B:B's parent is A.
- Move to A (start node).

So, the final path is: A → B → D → F

BFS explores nodes in a level-by-level manner, ensuring that the shortest path is found. It uses a queue to keep track of nodes to be explored, and the parent dictionary helps in reconstructing the path from the goal back to the start.

Depth-First Search (DFS)

Why is it called "Depth-First Search"?

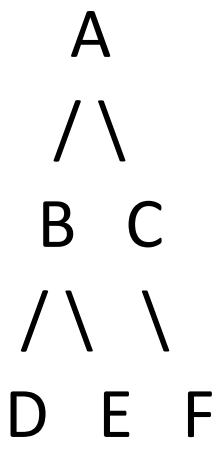
- **Depth** refers to the algorithm's focus on exploring **as far down a branch or path as possible** before backtracking to explore other branches.

1. The word **first** means that the algorithm prioritizes depth or **going as deep as possible into the search space** along one branch before returning to explore other branches.

- **Depth-first** literally means the algorithm searches **deeply into one branch** before considering other branches or neighboring nodes at the same level.

Depth-First Search (DFS)

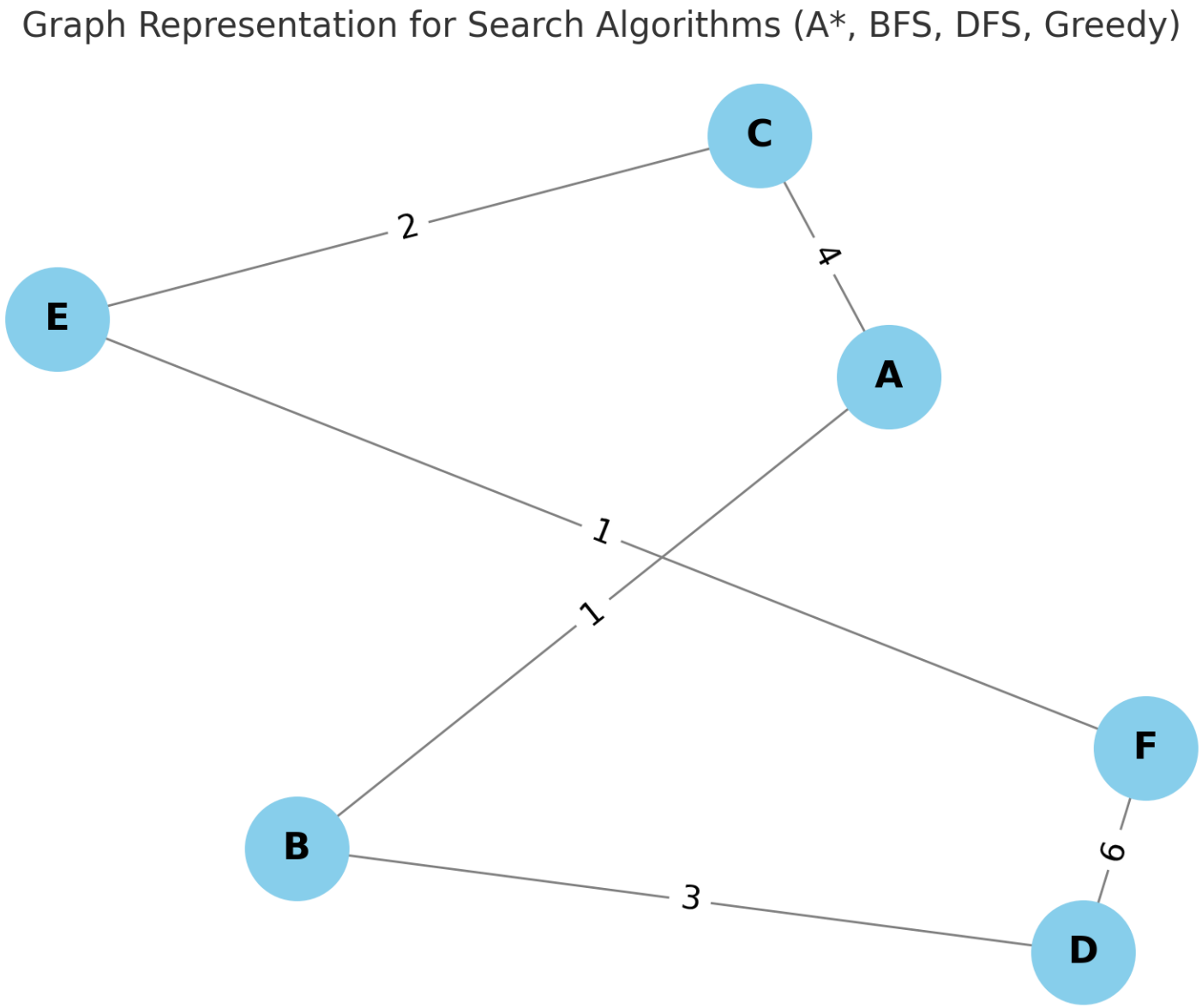
Visualization Example (Tree)



DFS explores the tree level by level, meaning:

- Start at A, go to B, then to D.
- After finishing D, backtrack to B, explore E.
- Then, backtrack to A, explore C, and go down to F.

Graph Representation for Search Algorithms



Depth-First Search (DFS)

Depth-First Search (DFS) algorithm step by step, using the example graph from earlier, and show the actions and updates to the stack, explored set, and the parent dictionary at each step.

- **Graph Setup:**
  - Nodes: A, B, C, D, E, F
  - Edges (connections): A-B, A-C, B-D, C-E, D-F, E-F
  - Start Node: A
  - Goal Node: F
- DFS Process: DFS uses a stack to explore nodes, and the principle is to go as deep as possible along one branch before backtracking.

Depth-First Search (DFS)

- Step 1: Initialize
  - Stack: [A]
  - Explored: {}
  - Parent: {A: None} (Since A is the starting point, it has no parent).

Depth-First Search (DFS)

- Step 2: Explore A
  - Pop A from the stack (exploring A).
  - Add A to the explored set.
  - Push A's neighbors (B, C) to the stack (last in, first out).
    - Stack: [B, C]
    - Explored: {A}
    - Parent: A: None,
      B: A,
      C: A

Depth-First Search (DFS)

- Step 3: Explore C
  - Pop C from the stack.
  - Add C to the explored set.
  - Push C's neighbor (E) to the stack.
    - Stack: [B, E]
    - Explored: {A, C}
    - Parent: A: None,
      B: A,
      C: A,
      E: C

Depth-First Search (DFS)

- Step 4: Explore E
  - Pop E from the stack.
  - Add E to the explored set.
  - Push E's neighbor (F) to the stack.
    - Stack: [B, F]
    - Explored: {A, C, E}
    - Parent: A: None,
      B: A,
      C: A,
      E: C,
      F: E

Depth-First Search (DFS)

- Step 5: Explore F (Goal Reached)
  - Pop F from the stack.
  - Add F to the explored set.
  - Since F is the goal, DFS terminates.
    - Stack: [B] (B is still in the stack, but we stop since the goal is reached).
    - Explored: {A, C, E, F}
    - Parent (no changes since we reached the goal):

      A: None,
      B: A,
      C: A,
      E: C,
      F: E

Depth-First Search (DFS)

- Step 6: Reconstruct the Path

Now, we reconstruct the path from the goal (F) back to the start (A) using the parent dictionary.

  - Start at F: F's parent is E.
  - Move to E: E's parent is C.
  - Move to C: C's parent is A.
  - Move to A (start node).

So, the final path is: A → C → E → F

DFS explores as far down one branch as possible (in this case, A → C → E → F) before backtracking. It uses a stack to keep track of nodes, and the parent dictionary helps in reconstructing the final path once the goal is reached.

Lecture Overview

- **Problem Solving via Search:** Uninformed search algorithms (BFS, DFS).
- **Heuristic Search:** Informed search algorithms (A*, greedy search).
- **Optimization Problems:** Hill climbing, Simulated annealing, Genetic algorithms.

Heuristic search

- **Heuristic search** is a search strategy in artificial intelligence that uses a heuristic function to guide the search process toward a solution more efficiently. The heuristic function provides an estimate of how close a given state is to the goal, helping the algorithm prioritize which paths to explore first. This makes the search process more directed and often faster, especially in large search spaces, compared to uninformed search methods like BFS or DFS.

Heuristic search

- Key Features:
  - **Guided by heuristics:** A heuristic function, $h(n)$, estimates the cost or distance from a given node n to the goal.
  - **Goal-oriented:** The search focuses on nodes that appear to be closer to the goal, based on the heuristic value.
  - Examples of Heuristic Search Algorithms:
    - **A*:** Combines actual cost $g(n)$ and heuristic $h(n)$ to find the optimal solution.
    - **Greedy Best-First Search:** Uses only the heuristic $h(n)$ to expand the node that appears closest to the goal.

Heuristic function

- The heuristic values in a Greedy Best-First Search/A* Search are typically based on a problem-specific function that estimates the cost from a given node to the goal node. These heuristic values are not calculated by the search algorithm itself; rather, they are predefined based on some understanding or approximation of the problem.

Heuristic function

To clarify, let's break down how such heuristic values could be determined, using a few common techniques:

- **Straight-Line (Euclidean) Distance**

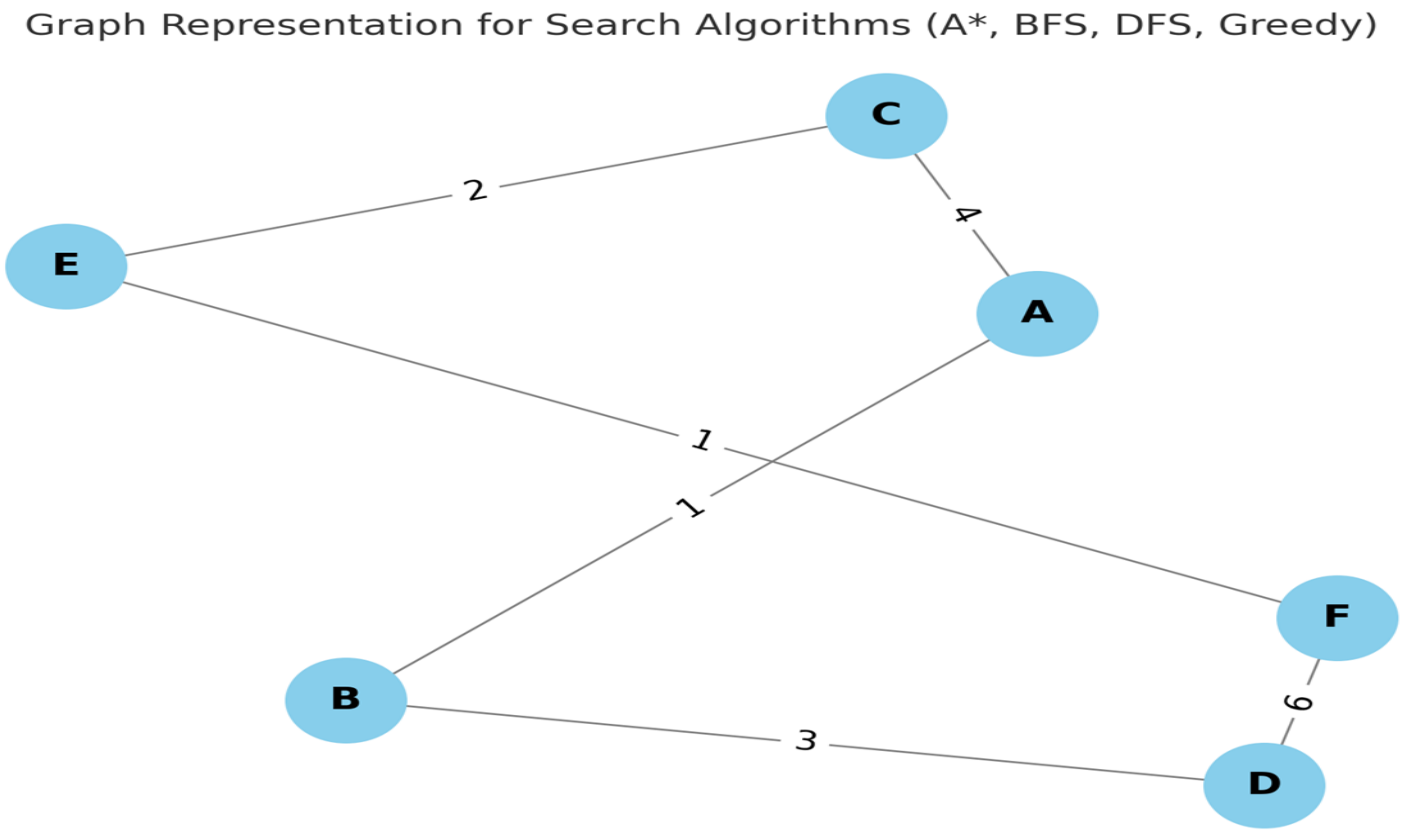
This method is often used in pathfinding problems where nodes represent points in a 2D, or 3D space (like cities on a map). The heuristic values represent the straight-line distance (as the crow flies) from a node to the goal node.

  - For example: If you have nodes representing cities and edges representing roads, you could use the straight-line distance between each city and the goal as the heuristic value.

Heuristic function

- Formula for Euclidean Distance:

$$h(n) = \sqrt{\left\{\left(x_{goal} - x_{current}\right)^2 + \left(y_{goal} - y_{current}\right)^2\right\}}$$



Heuristic function

- **Example:**

Let's say the nodes represent coordinates in a 2D grid, and the goal node F is at position (5,7). For each node, the heuristic value h(n) could be the Euclidean distance to node F.

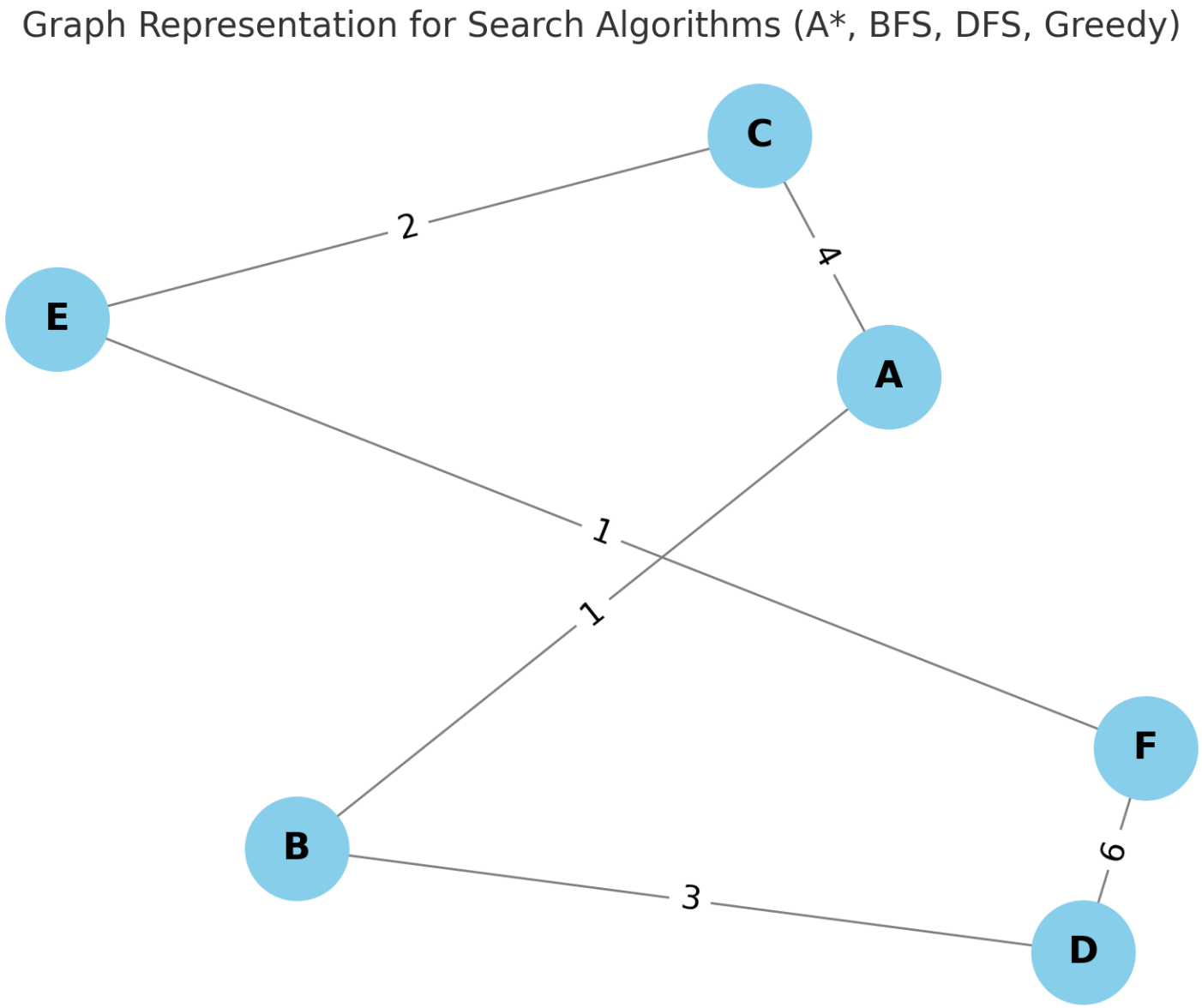
- For node A at (1,1):

$$h(A) = \sqrt{\{(5-1)^2 + (7-1)^2\}}=7.2$$

- For node B at (3,4):

$$h(B) = \sqrt{\{(5-3)^2 + (7-4)^2\}}=3.6$$

Graph Representation for Search Algorithms



Greedy Best-First Search

Greedy Best-First Search step by step using the same graph example and show the actions, updates to the priority queue, explored set, and parent dictionary at each step.

Graph Setup:

- Nodes: A, B, C, D, E, F
- Edges (connections): A-B, A-C, B-D, C-E, D-F, E-F
- Start Node: A
- Goal Node: F

Greedy Best-First Search

Heuristic Values $h(n)$:

In Greedy Best-First Search, we use a heuristic function $h(n)$ to estimate the distance from any node to the goal node F. The heuristic function drives the search process, as it always picks the node with the lowest $h(n)$ value from the priority queue.

Let's assign heuristic values to the nodes: $h(A)$=7,$h(B)$=6, $h(C)$=4, $h(D)$=3, $h(E)$=2, $h(F)$=0 (since F is the goal)

Greedy Best-First Search

Why Did I Assign Heuristic Values Like $h(A)$=7, $h(B)$=6, etc.?

These heuristic values were assigned as examples based on the assumption that the nodes represent locations in some kind of abstract space, and the heuristic values reflect how "far" each node is from the goal node. In this case:

$h(A)$=7: Node A is relatively far from the goal.

$h(B)$=6: Node B is slightly closer to the goal.

$h(F)$=0: Since F is the goal, the heuristic value $h(F)$ is 0 because there is no distance left to travel.

In a real-world problem, these values would be computed using a heuristic function like Euclidean distance or Manhattan distance, depending on the type of problem you're solving.

Greedy Best-First Search

Greedy Search Process:

Greedy search uses a priority queue ordered by the heuristic value $h(n)$ to guide its exploration. It picks the node that seems closest to the goal according to $h(n)$, ignoring the actual cost to reach the node.

Greedy Best-First Search

Step 1: Initialize

- Priority Queue: [(A, h(A) = 7)](Insert the start node with its heuristic value)
- Explored: {}
- Parent: {A: None} (Since A is the starting point, it has no parent).

Greedy Best-First Search

Step 2: Explore A

- Dequeue A from the priority queue (lowest $h(A)$=7).
- Add A to the explored set.
- Enqueue A's neighbors (B, C) into the priority queue with their heuristic values.
  - Priority Queue: [(C, h(C) = 4), (B, h(B) = 6)](C has lower heuristic, so it is placed before B)
  - Explored: {A}
  - Parent: A: None,
    B: A,
    C: A

Greedy Best-First Search

Step 3: Explore C

- Dequeue C from the priority queue (lowest $h(C)$=4).
- Add C to the explored set.
- Enqueue C's neighbor (E) into the priority queue with its heuristic value.
  - Priority Queue: [(E, h(E) = 2), (B, h(B) = 6)]
  - Explored: {A, C}
  - Parent: A: None,
    B: A,
    C: A,
    E: C

Greedy Best-First Search

Step 4: Explore E

- Dequeue E from the priority queue (lowest $h(E)$=2).
- Add E to the explored set.
- Enqueue E's neighbor (F) into the priority queue with its heuristic value.
  - Priority Queue: [(F, h(F) = 0), (B, h(B) = 6)]
  - Explored: {A, C, E}
  - Parent: A: None,
    B: A,
    C: A,
    E: C,
    F: E

Greedy Best-First Search

Step 5: Explore F (Goal Reached)

- Dequeue F from the priority queue (lowest $h(F)$=0).
- Add F to the explored set.
- Since F is the goal, Greedy Search terminates.
  - Priority Queue: [(B, h(B) = 6)] (B is still in the queue, but we stop since the goal is reached).
  - Explored: {A, C, E, F}
  - Parent (no changes since we reached the goal):

    A: None,

    B: A,

    C: A,

    E: C,

    F: E

Greedy Best-First Search

Step 6: Reconstruct the Path

Now, we reconstruct the path from the goal (F) back to the start (A) using the parent dictionary.

- Start at F:F's parent is E.
- Move to E:E's parent is C.
- Move to C:C's parent is A.
- Move to A (start node).

So, the final path is: A → C → E → F

Greedy Best-First Search

Greedy Best-First Search uses a priority queue based solely on the heuristic $h(n)$, which estimates the distance to the goal. It always picks the node that appears to be closest to the goal based on $h(n)$, but it doesn't guarantee an optimal solution, as it ignores the actual cost of reaching the nodes. The parent dictionary is used to reconstruct the path from the goal back to the start.

A* Search

A* Search step by step, just like we did for BFS, DFS, and Greedy search. A* search combines the actual cost to reach a node $g(n)$ and a heuristic estimate $h(n)$ of the cost to reach the goal from that node. The algorithm uses a priority queue where nodes are prioritized based on $f(n)=g(n)+h(n)$.

Graph Setup:

Nodes: A, B, C, D, E, F

Edges (connections with costs):A-B (1), A-C (4),B-D (3), C-E (2),D-F (6), E-F (1)

Start Node: A

Goal Node: F

A* Search

Heuristic Values $h(n)$: We use a heuristic $h(n)$ to estimate the cost from a node to the goal F:

$h(A)$=7,

$h(B)$=6,

$h(C)$=4,

$h(D)$=3,

$h(E)$=2,

$h(F)$=0(since F is the goal)

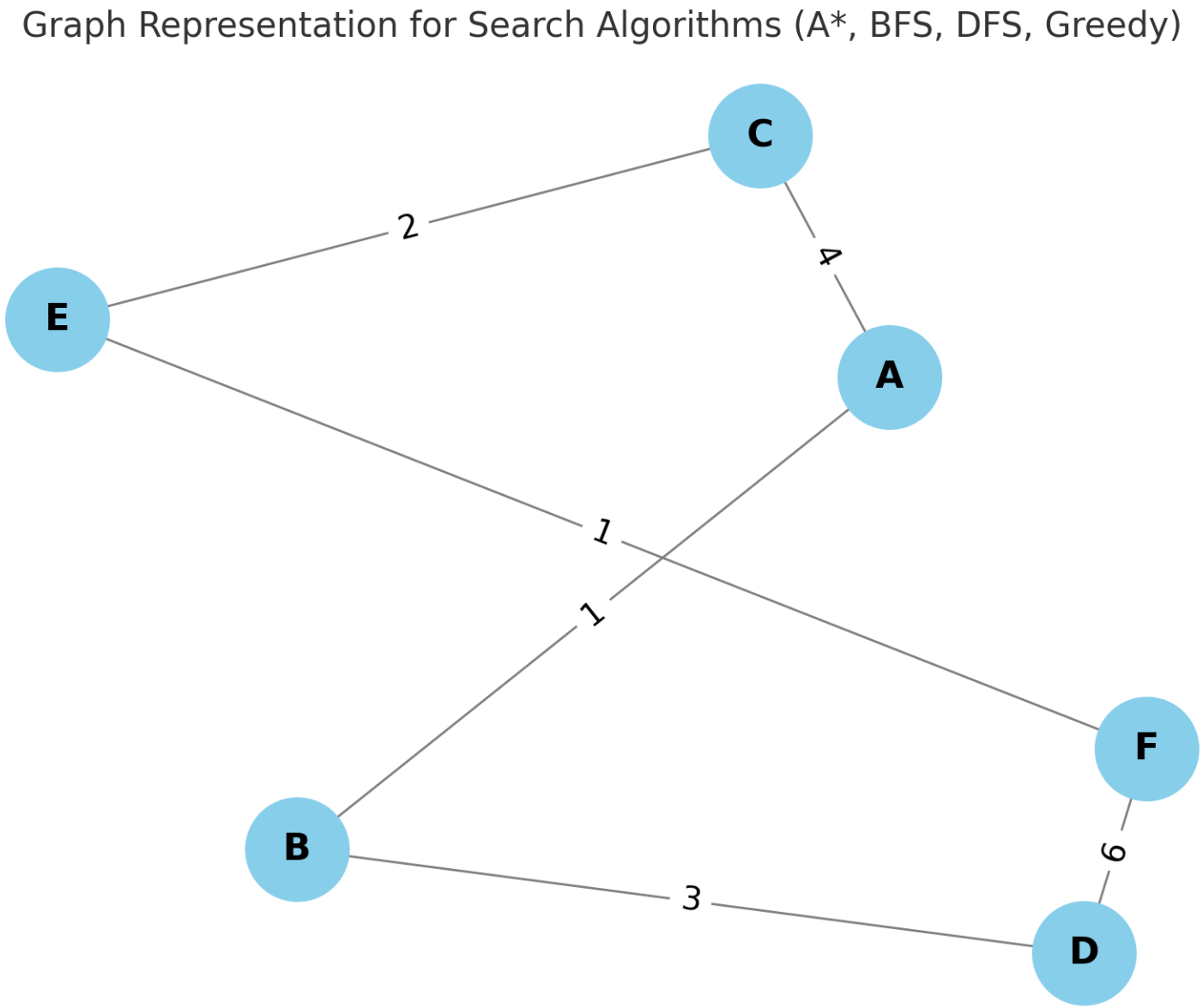
A* Search Process: A* uses a priority queue ordered by the evaluation function $f(n)=g(n)+h(n)$. It always expands the node with the lowest $f(n)$ value.

A* Search

A* Search Process: A* uses a priority queue ordered by the evaluation function $f(n)=g(n)+h(n)$. It always expands the node with the lowest $f(n)$ value.

- $g(n)$ is the actual cost from the start node to the current node.
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal node.
- $f(n)$ is the estimated total cost of the path through the current node.

Graph Representation for Search Algorithms



A* Search

Step 1: Initialize

- Priority Queue: [(A, f(A) = g(A) + h(A) = 0 + 7 = 7)]
  - Start with node A and its evaluation function $f(A)$.
- Explored: {}
- Parent: {A: None} (A has no parent since it's the start).

A* Search

Step 2: Explore A

- Dequeue A from the priority queue (lowest $f(A)$=7).
- Add A to the explored set.
- Enqueue A's neighbors (B and C) into the priority queue with their $f(n)$ values.

  For B: $g(B)$=$g(A)$+cost($A \rightarrow B$)=0+1

  $f(B)$=$g(B)$+$h(B)$=1+6=7

  For C: $g(C)$=$g(A)$+cost($A \rightarrow C$)=0+4=4

  $f(C)$=$g(C)$+$h(C)$=4+4=8

  - Priority Queue: [(B, f(B) = 7), (C, f(C) = 8)]
  - Explored: {A}
  - Parent: A: None,
    B: A,
    C: A

A* Search

Step 3: Explore B

- Dequeue B from the priority queue (lowest $f(B)$=7).
- Add B to the explored set.
- Enqueue B's neighbor (D) into the priority queue with its $f(n)$ value.

  For D: $g(D)=g(B)+\text{cost}(B \rightarrow D)=1+3=4$

  $f(D)=g(D)+h(D)=4+3=7$

  - Priority Queue: [(D, f(D) = 7), (C, f(C) = 8)]
  - Explored: {A, B}
  - Parent:

    A: None,

    B: A,

    C: A,

    D: B

A* Search

Step 4: Explore D

- Dequeue D from the priority queue (lowest $f(D)$=7).
- Add D to the explored set.
- Enqueue D's neighbor (F) into the priority queue with its $f(n)$ value.

  For F: $g(F)$=$g(D)$+cost($D \rightarrow F$)=4+6=10

  $f(F)$=$g(F)$+$h(F)$=10+0=10

  - Priority Queue: [(C, f(C) = 8), (F, f(F) = 10)]
  - Explored: {A, B, D}
  - Parent: A: None,
    B: A,
    C: A,
    D: B
    F: D

A* Search

Step 5: Explore C

- Dequeue C from the priority queue (lowest $f(C)$=8).
- Add C to the explored set.
- Enqueue C's neighbor (E) into the priority queue with its $f(n)$ value.

  For E: $g(E)=g(C)+\text{cost}(C \rightarrow E)=4+2=6$

  $f(E)=g(E)+h(E)=6+2=8$

  - Priority Queue: [(E, f(E) = 8), (F, f(F) = 10)]
  - Explored: {A, B, C, D}
  - Parent: : A: None,
    B: A,
    C: A,
    D: B
    E: C
    F: D

A* Search

Step 6: Explore E

- Dequeue E from the priority queue (lowest $f(E)$=8).
- Add E to the explored set.
- Enqueue E's neighbor (F) into the priority queue with its updated $f(n)$ value.

  For F: $g(F)$=$g(E)$+cost($E \rightarrow F$)=6+1=7

  $f(F)$=$g(F)$+$h(F)$=7+0=7

  - Priority Queue: [(F, f(F) = 7), (F, f(F) = 10)](F appears twice with different $f(n)$ values, but we will use the lower one)
  - Explored: {A, B, C, D, E}
  - Parent: A: None,
    B: A,
    C: A,
    D: B
    E: C
    F: E

A* Search

Step 7: Explore F (Goal Reached)

- Dequeue F from the priority queue (lowest $f(F)$=7).
- Add F to the explored set.
- Since F is the goal, A* terminates.
  - Priority Queue: [(F, f(F) = 10)] (the other instance of F remains, but we stop here).
  - Explored: {A, B, C, D, E, F}
  - Parent (no changes since we reached the goal):

    A: None,

    B: A,

    C: A,

    D: B

    E: C

    F: E

A* Search

Step 8: Reconstruct the Path

Now, we reconstruct the path from the goal (F) back to the start (A) using the parent dictionary.

- Start at F:F's parent is E.
- Move to E:E's parent is C.
- Move to C:C's parent is A.
- Move to A (start node).

So, the final path is: A → C → E → F

A* Search (Practice Example)

Defining the Graph with 2D Coordinates

We will place the nodes A, B, C, D, E, and F on a 2D plane with the following coordinates:

$A(0,0)$

$B(1,2)$

$C(4,2)$

$D(1,5)$

$E(4,5)$

$F(6,6)$ (Goal)

A* Search (Practice Example)

Edges with Costs:

Let's use the following edges between nodes with given costs:

$A \rightarrow B$ (Cost = 2)

$A \rightarrow C$ (Cost = 5)

$B \rightarrow D$ (Cost = 3)

$C \rightarrow E$ (Cost = 2)

$D \rightarrow F$ (Cost = 6)

$E \rightarrow F$ (Cost = 1)

A* Search (Practice Example)

Graph Visualization:

The graph shows the nodes and edges, along with the costs associated with each edge. Additionally, the heuristic values $h(n)$, based on the Euclidean distance from each node to the goal node F, are as follows:

$h(A)$=8.49

$h(B)$=6.40

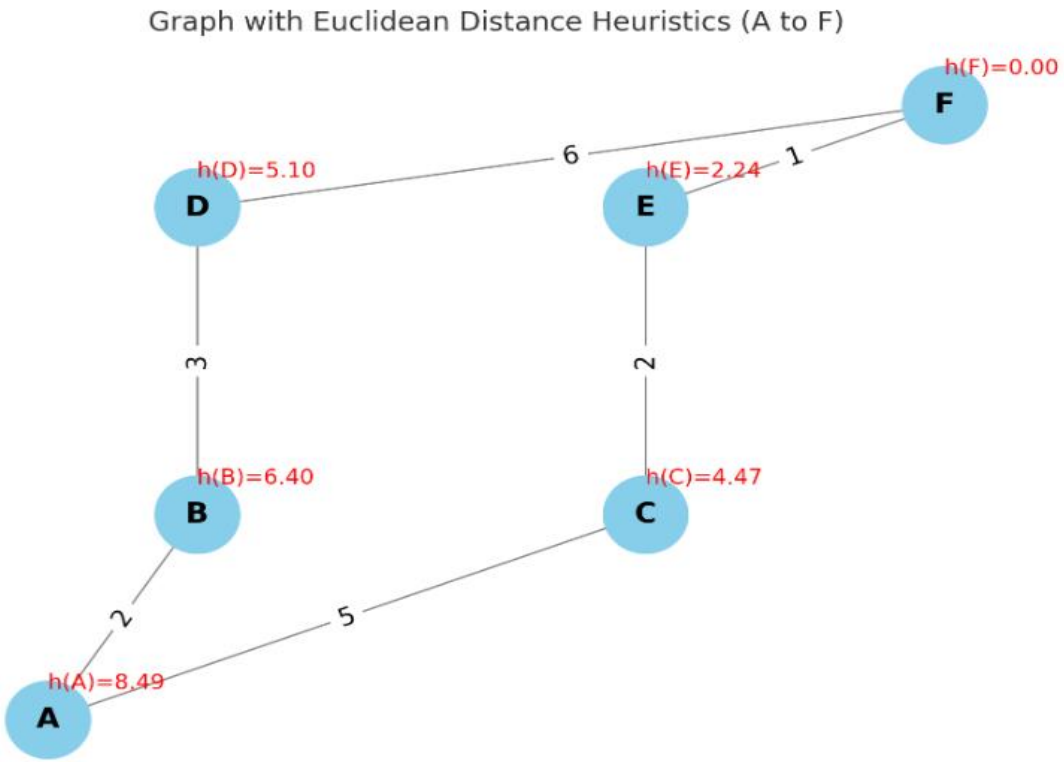
$h(C)$=4.47

$h(D)$=5.10

$h(E)$=2.24

$h(F)$=0.00

(since F is the goal node)



A* Search (Practice Example)

Step-by-Step Execution of A*:

1. Start at Node A:

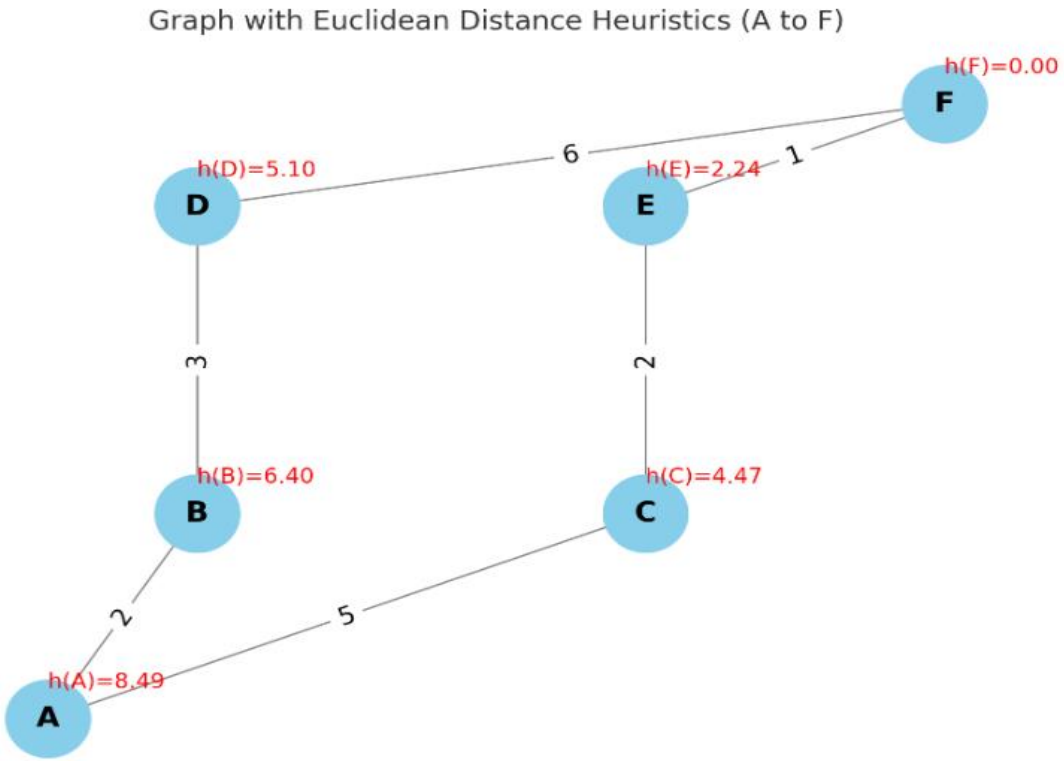
$g(A)$=0 (cost to reach A)

$h(A)$=8.49 (Euclidean distance from A to F)

$f(A)$=$g(A)$+$h(A)$=0+8.49

=8.49

Queue: [A]



A* Search (Practice Example)

Step-by-Step Execution of A*:

2. Expand A:

- Neighbors: B and C

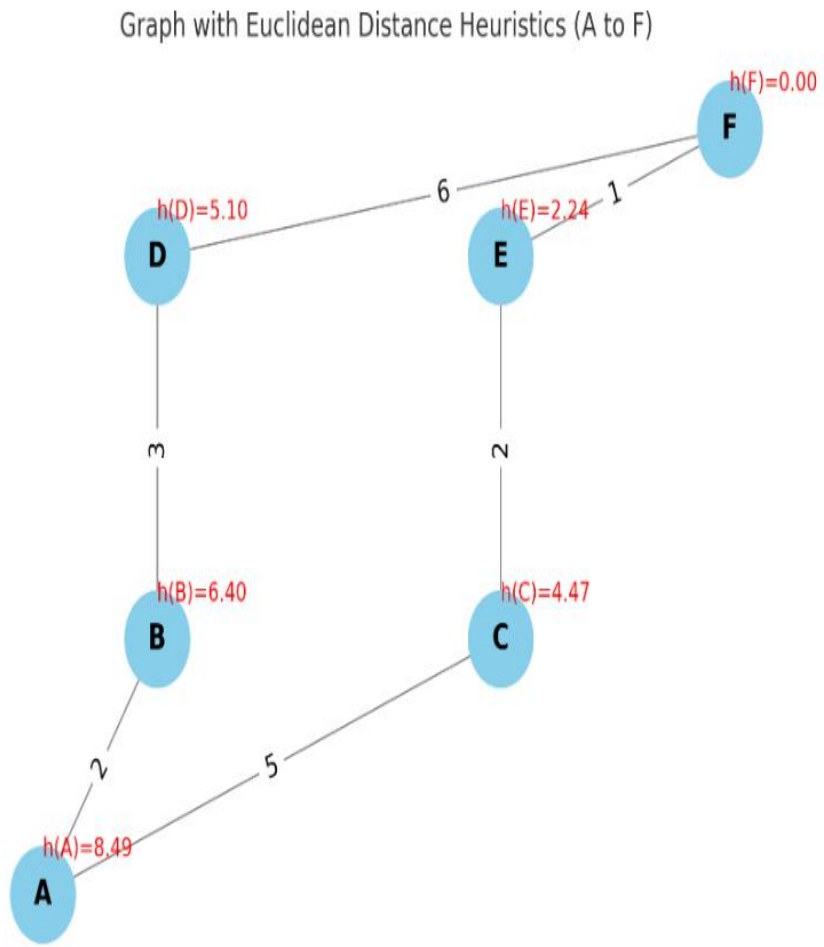
For B:

- $g(B)$=$g(A)$+cost($A \rightarrow B$)=0+2=2
- $h(B)$=6.40
- $f(B)$=$g(B)$+$h(B)$=2+6.40=8.40

For C:

- $g(C)$=$g(A)$+cost($A \rightarrow C$)=0+5=5
- $h(C)$=4.47
- $f(C)$=$g(C)$+$h(C)$=5+4.47=9.47

Queue: [B (f = 8.40), C (f = 9.47)]



A* Search (Practice Example)

Step-by-Step Execution of A*:

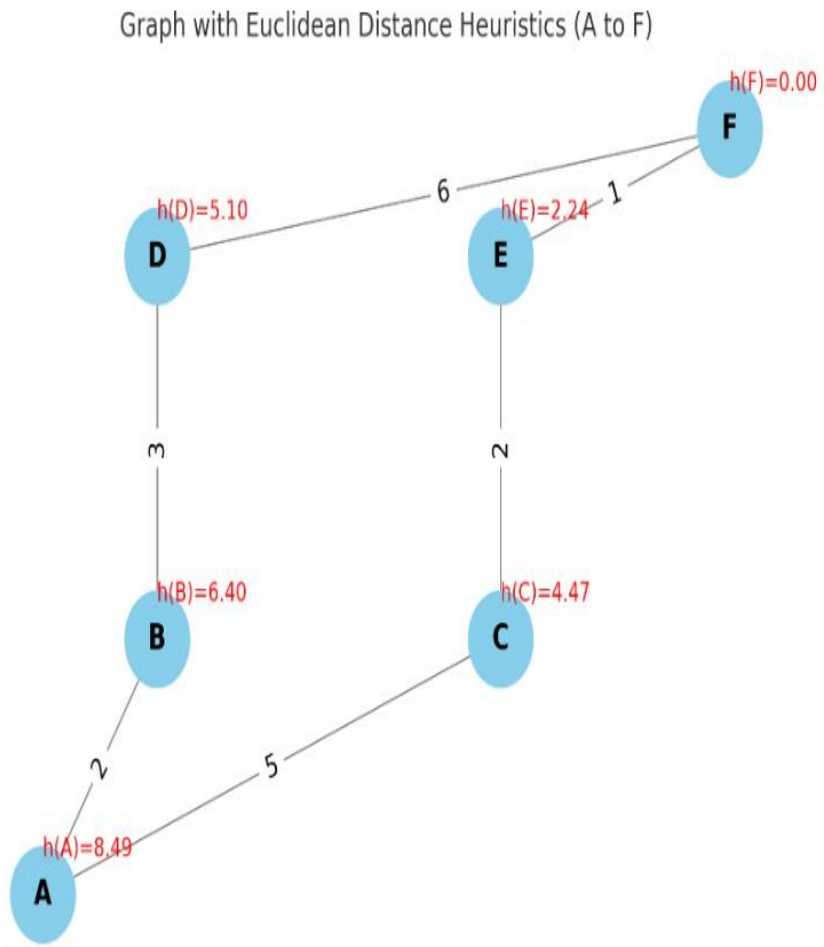
3. Expand B (lowest f-value):

- Neighbors: A (already visited), D

For D:

- $g(D)=g(B)+cost(B \rightarrow D)=2+3=5$
- $h(D)=5.10$
- $f(D)=g(D)+h(D)=5+5.10=10.10$

Queue [C (f = 9.47), D (f = 10.10)]



A* Search (Practice Example)

Step-by-Step Execution of A*:

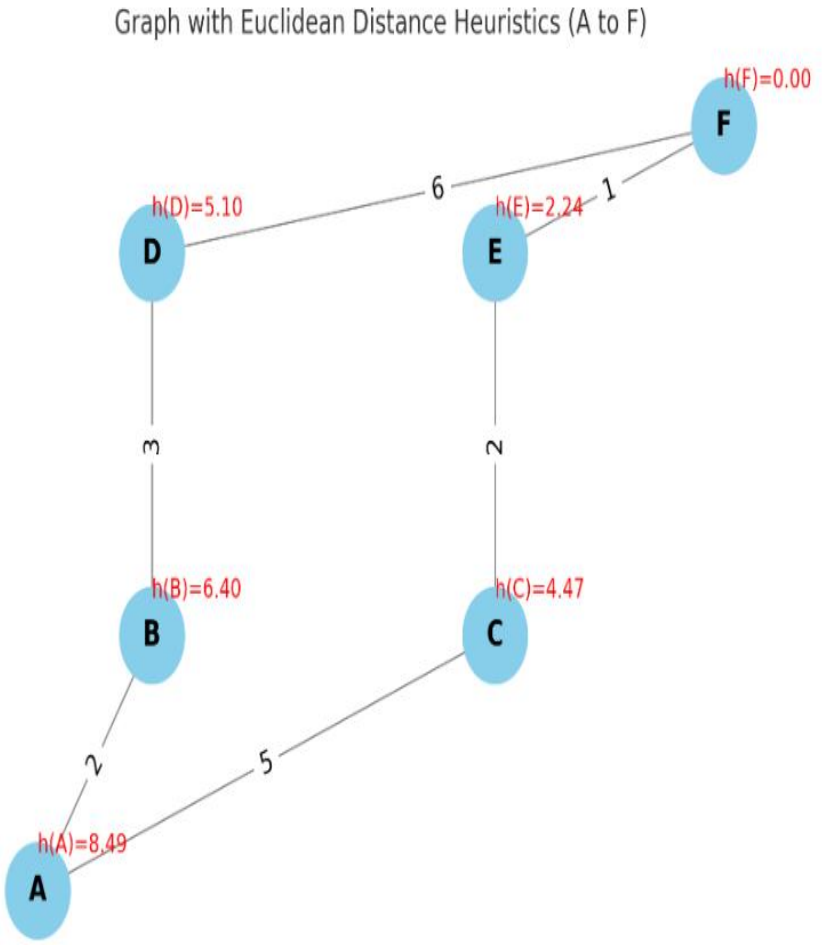
4. Expand C (lowest f-value):

- Neighbors: A (already visited), E

For E:

- $g(E)$=$g(C)$+cost($C \rightarrow E$)=5+2=7
- $h(E)$=2.24
- $f(E)$=$g(E)$+$h(E)$=7+2.24=9.24

Queue: [E (f = 9.24), D (f = 10.10)]



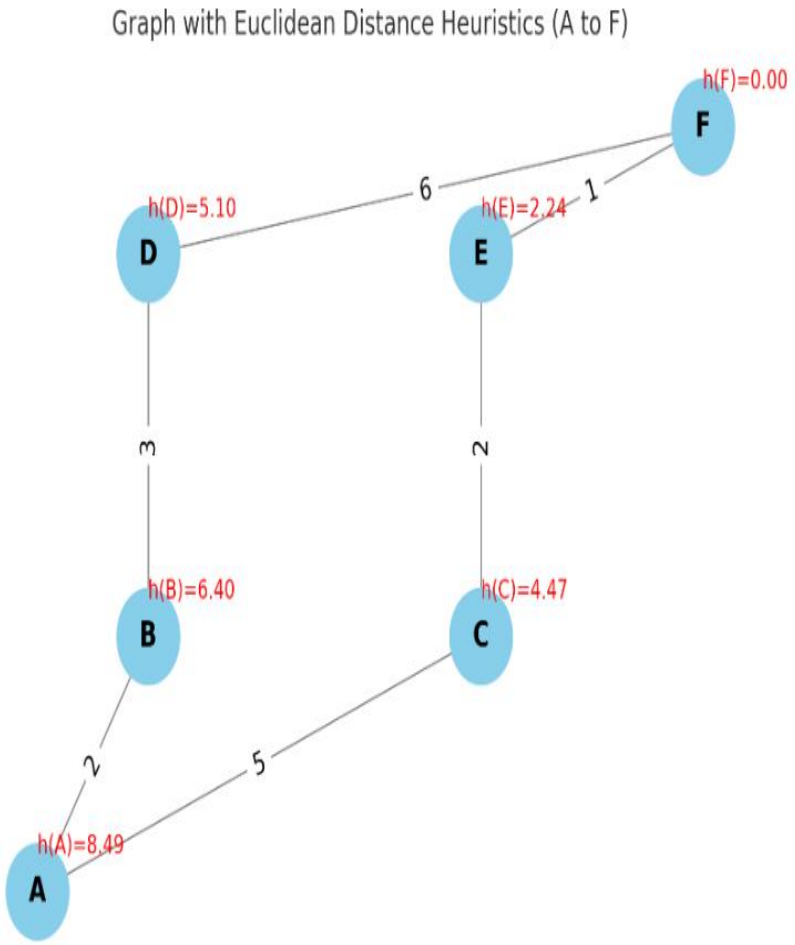
A* Search (Practice Example)

Step-by-Step Execution of A*:

5. Expand E:

- Neighbors: C (already visited), F
- **For F:**
- g(F)=g(E)+cost(E→F)=7+1=8
- $h(F)$=0 (since F is the goal)
- $f(F)=g(F)+h(F)$=8+0=8

Queue: [F (f = 8), D (f = 10.10)]



A* Search (Practice Example)

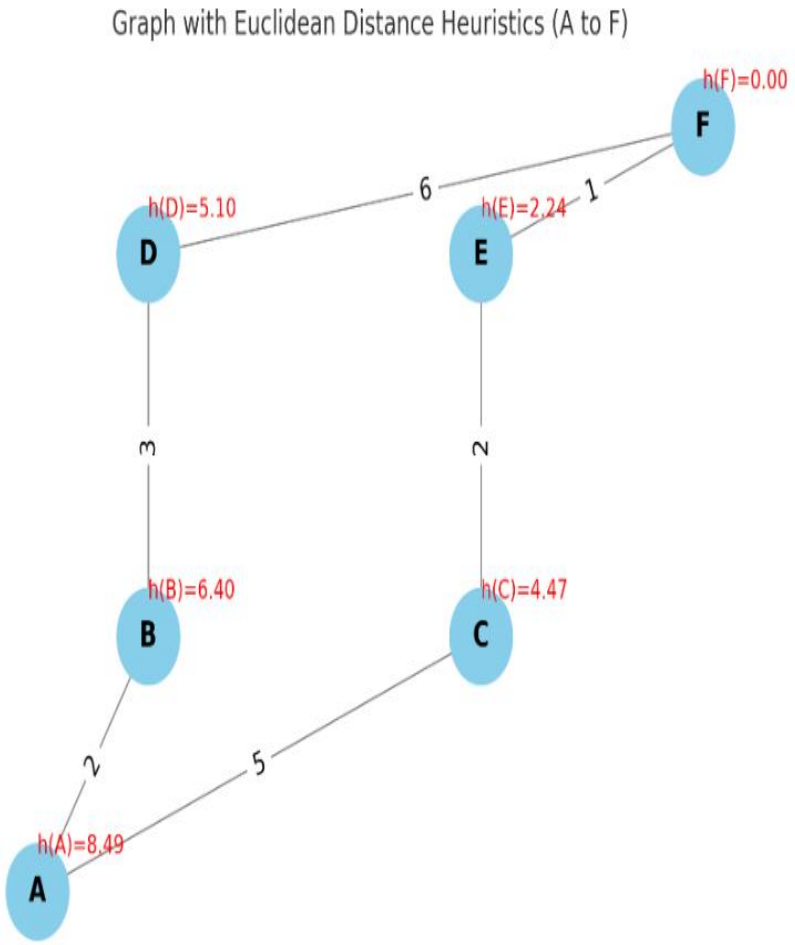
Step-by-Step Execution of A*:

6. **Expand F:**

- The goal node F is reached with a total cost of 8.

Path Found by A*:

The path is A → C → E → F with a total cost of 8.



A* Search (Practice Example)

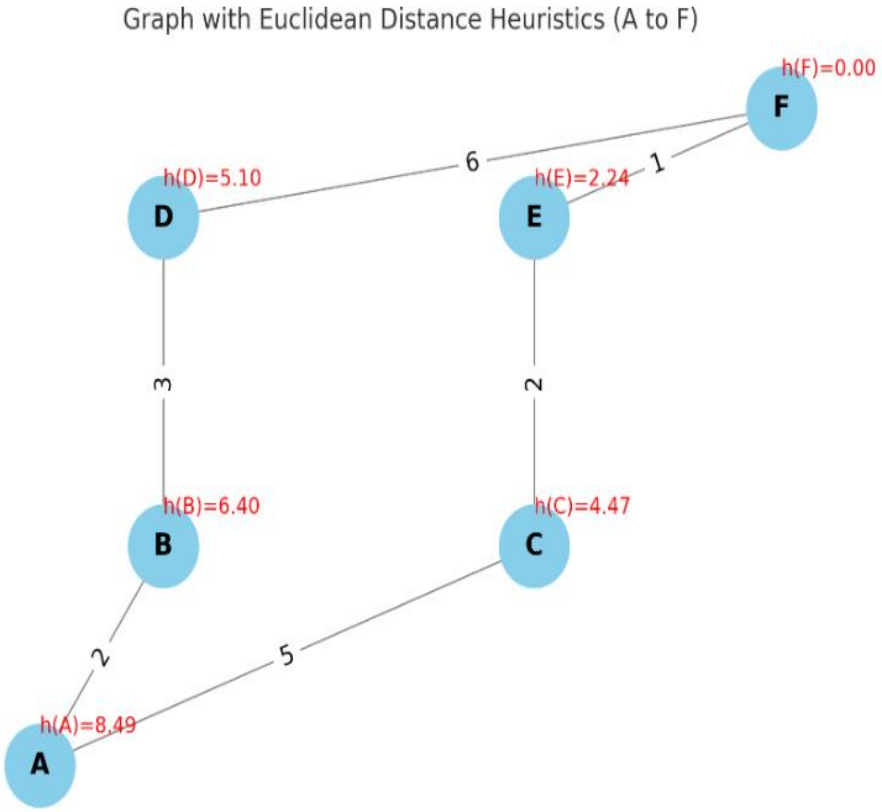
Reconstruct the Path:

Once the goal node F is reached, we can trace back the path using the parent dictionary.

- F's parent is E.
- E's parent is C.
- C's parent is A.

Path Found by A*:

The path is A → C → E → F with a total cost of 8.

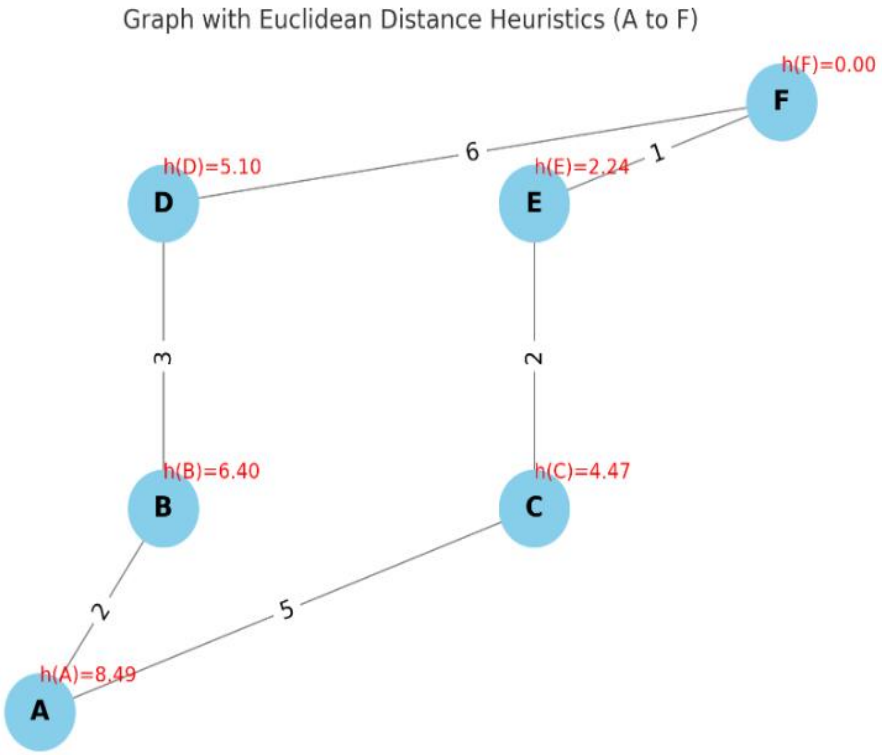


Greedy Best-First Search (Practice Example)

Greedy Best-First Search only considers the heuristic values $h(n)$and selects the node with the lowest heuristic value at each step.

1. Start at Node A:
   - $h(A)$=8.49

Queue: [A]



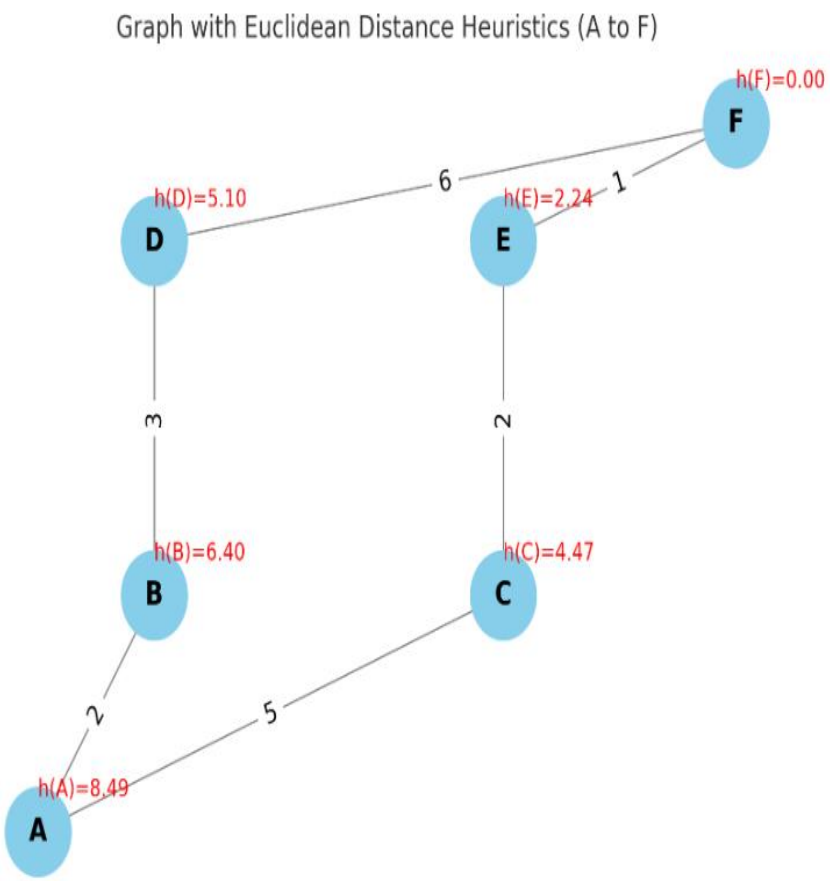
Greedy Best-First Search (Practice Example)

2. Expand A:

Neighbors: B and C

- For B: $h(B)$=6.40
- For C: $h(C)$=4.47

Queue: [C (h = 4.47), B (h = 6.40)]



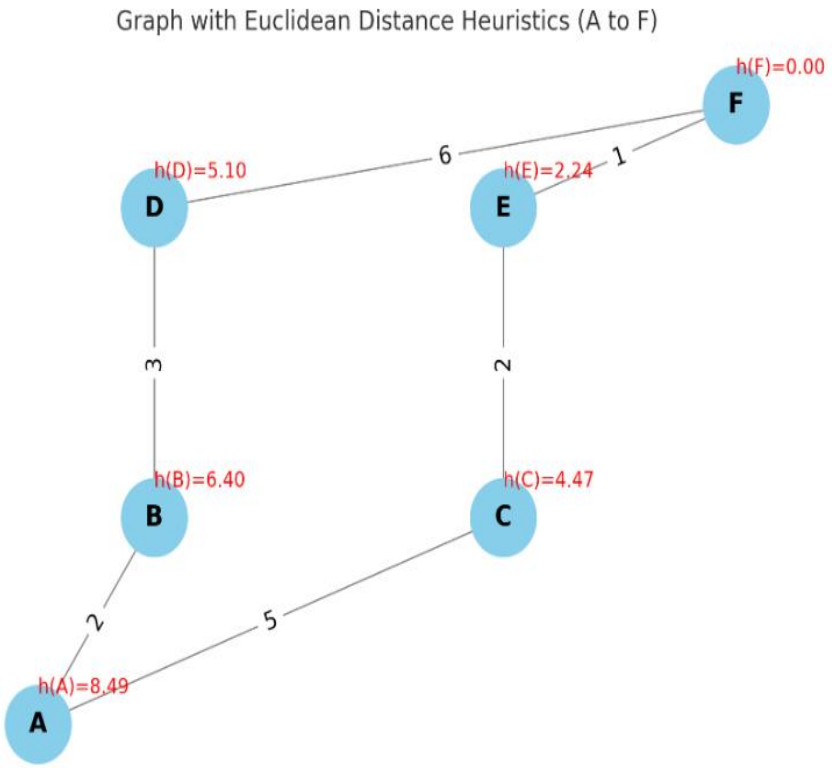
Greedy Best-First Search (Practice Example)

3. Expand C (lowest h-value):

Neighbors: A (already visited), E

- For E: h(E)=2.24

Queue: [E (h = 2.24), B (h = 6.40)]



Greedy Best-First Search (Practice Example)

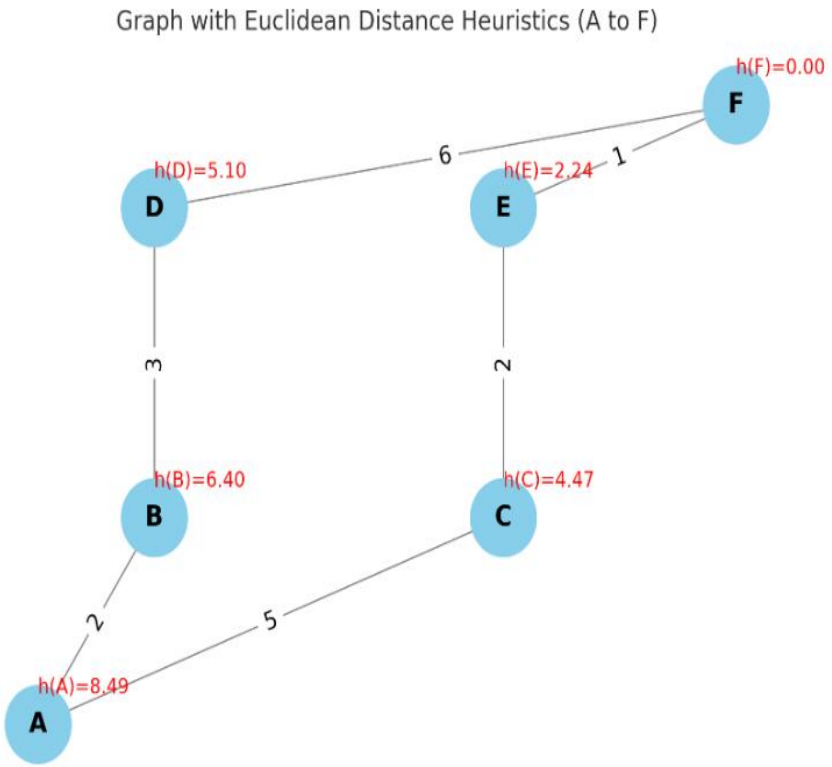
4. Expand E:

Neighbors: C (already visited), F

For F:

- h(F)=0

Queue: [F (h = 0), B (h = 6.40)]



Greedy Best-First Search (Practice Example)

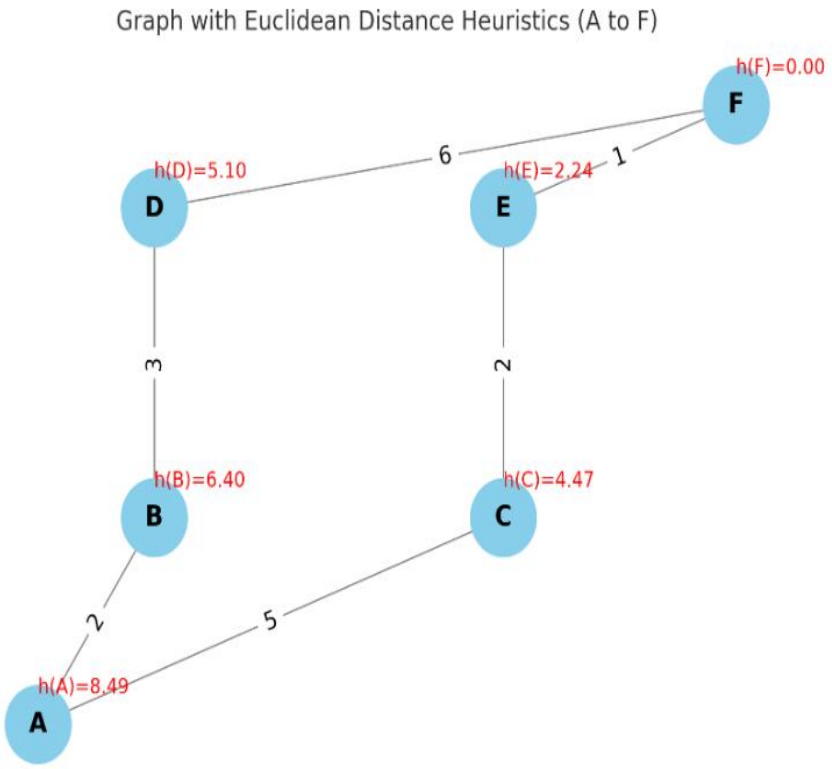
4. Expand F:

The goal node F is reached.

Path Found by Greedy Best-First Search:

The path is A → C → E → F

(same as A*), but Greedy ignores the total cost and only uses heuristic values.



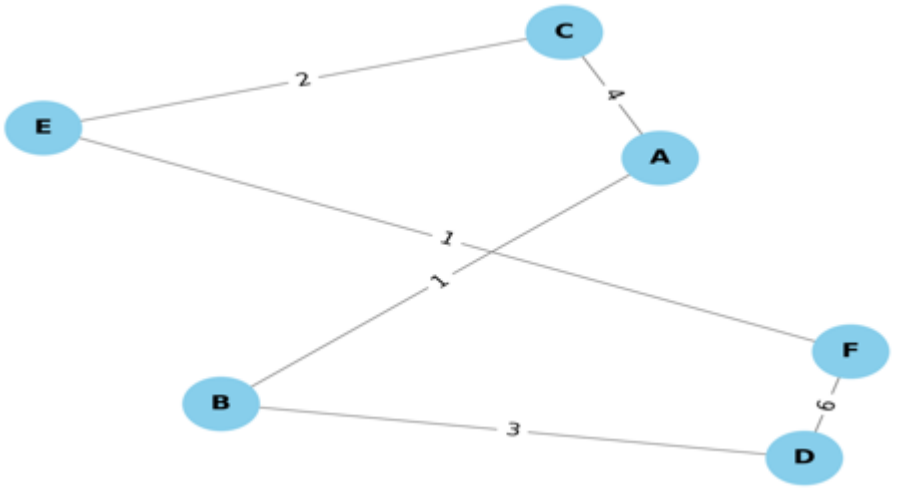
Practical Example (Maze Problem)

Maze Grid Conversion:

We can map this graph into a 5x5 maze grid where each letter represents a node, and the paths between them are represented as open cells (.) in the grid. Walls in the maze can be represented as #.

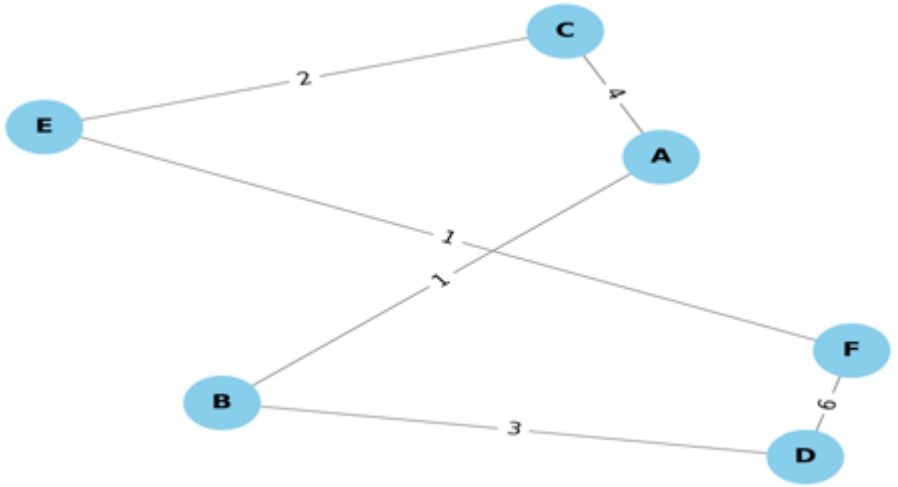
Here's how the grid could look:

# # # # #
A . B
. # .
C # D
. # .
E . F
# # # # #



Practical Example (Maze Problem)

```
#####  #
#A.B   #
#.  #  .  #
#C #D  #
#  .#  .  #
#E  .  F  #
#####   #
```

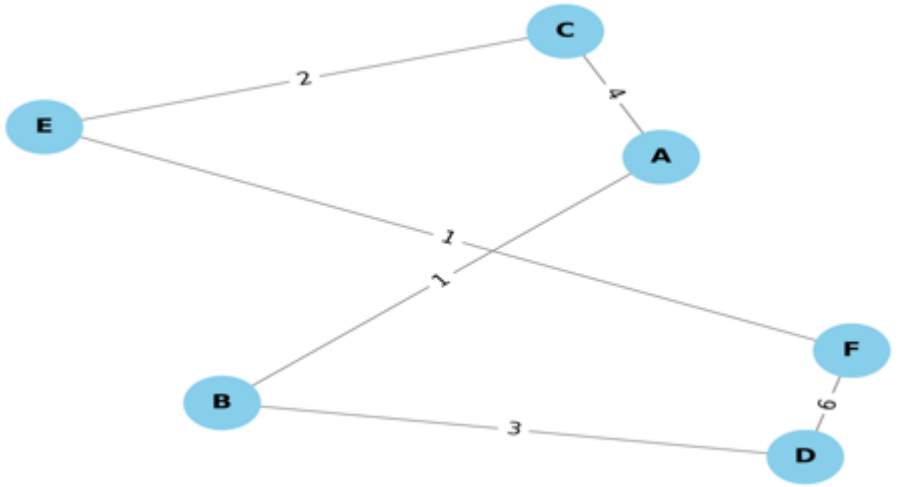


Explanation:

- Nodes A, B, C, D, E, F are placed at different grid positions.
- Connections (edges) between the nodes are represented as open paths (.).
  - There's an open path between A and B, B and D, A and C, C and E, D and F, and E and F.
- Walls (#) surround the nodes and the paths to form a maze-like structure.

Practical Example (Maze Problem)

```
##### #
#A.B  #
#. # . #
#C #D #
# . # . #
#E . F  #
#### #
```

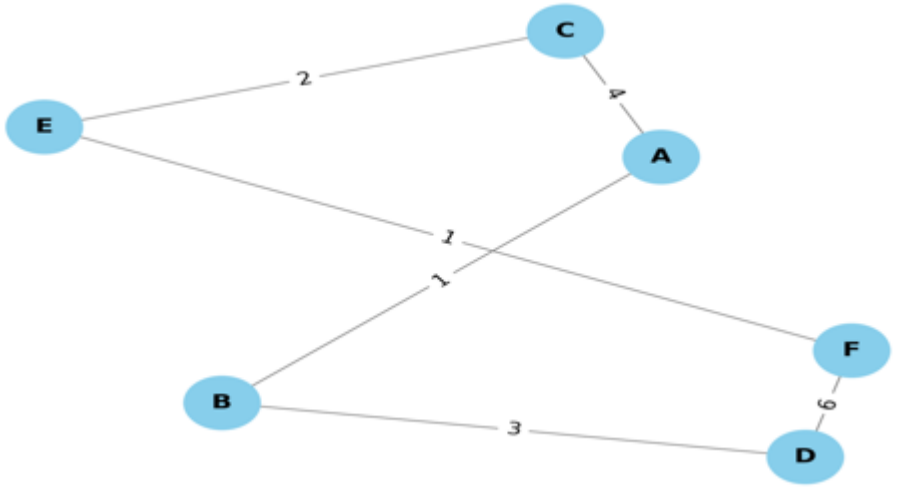


Key Points:

- A-B-C-D-E-F are interconnected, similar to how they were connected in the original graph.
- The rest of the grid is filled with walls to create a maze-like appearance.

Practical Example (Maze Problem)

```
##### #
#A.B  #
#.  #.  #
#C #D #
#.#.  #
#E.F  #
#### #
```



Using the Maze:

- Start at A and navigate through the open paths (.) to reach F.
- Follow the connections: A → B → D → F or A → C → E → F.

Lecture Overview

- **Problem Solving via Search:** Uninformed search algorithms (BFS, DFS).
- **Heuristic Search:** Informed search algorithms (A*, greedy search).
- **Optimization Problems:** Hill climbing, Simulated annealing, Genetic algorithms.

Hill Climbing Algorithm

Overview:

- **Hill Climbing** is a greedy local search algorithm that always moves in the direction of increasing value or decreasing cost, trying to reach the optimal solution as quickly as possible.
- It only considers neighboring nodes and moves to the one that seems the best in terms of minimizing the cost.
- However, Hill Climbing can get stuck in local optima, where no immediate neighbors offer a better solution, but the global optimum (goal) might still be far away.

Hill Climbing Algorithm

Hill Climbing Algorithm step by step for the same graph. In Hill Climbing, we aim to reach the goal node by always selecting the next node with the lowest heuristic value. This means that at each step, the algorithm chooses the neighbor that seems closest to the goal based on the heuristic values.

Hill Climbing Algorithm

Key Characteristics:

Greedy: At each step, it picks the best immediate option without considering future consequences.

Local Optimum: Hill Climbing can get stuck if it reaches a point where all neighboring nodes are worse, even if a better solution exists further away.

In this case, the algorithm reaches F but at a high cost (10). It doesn’t explore C or E, which might lead to a lower-cost path.

Hill Climbing Algorithm

Defining the Graph with 2D Coordinates

We will place the nodes A, B, C, D, E, and F on a 2D plane with the following coordinates:

$A(0,0)$

$B(1,2)$

$C(4,2)$

$D(1,5)$

$E(4,5)$

$F(6,6)$ (Goal)

Hill Climbing Algorithm

Edges with Costs:

Let's use the following edges between nodes with given costs:

$A \rightarrow B$ (Cost = 2)

$A \rightarrow C$ (Cost = 5)

$B \rightarrow D$ (Cost = 3)

$C \rightarrow E$ (Cost = 2)

$D \rightarrow F$ (Cost = 6)

$E \rightarrow F$ (Cost = 1)

Hill Climbing Algorithm

Graph Visualization:

The graph shows the nodes and edges, along with the costs associated with each edge. Additionally, the heuristic values $h(n)$, based on the Euclidean distance from each node to the goal node F, are as follows:

$h(A)$=8.49

$h(B)$=6.40

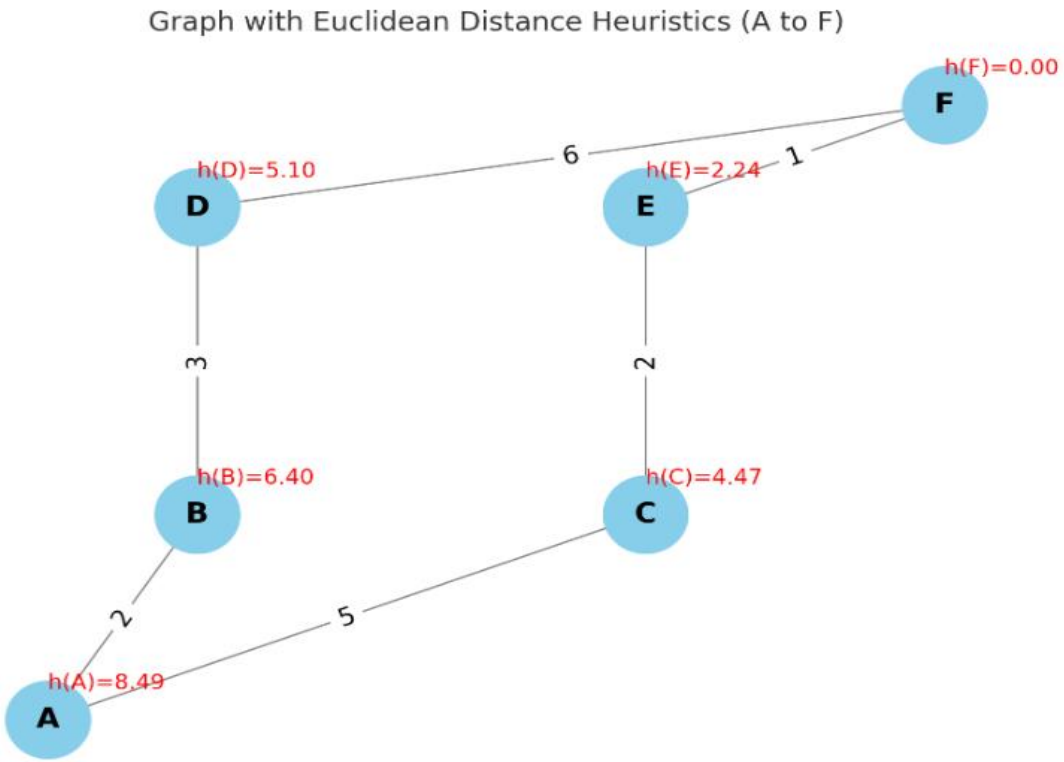
$h(C)$=4.47

$h(D)$=5.10

$h(E)$=2.24

$h(F)$=0.00

(since F is the goal node)



Hill Climbing Algorithm

Step-by-Step Execution of Hill Climbing:

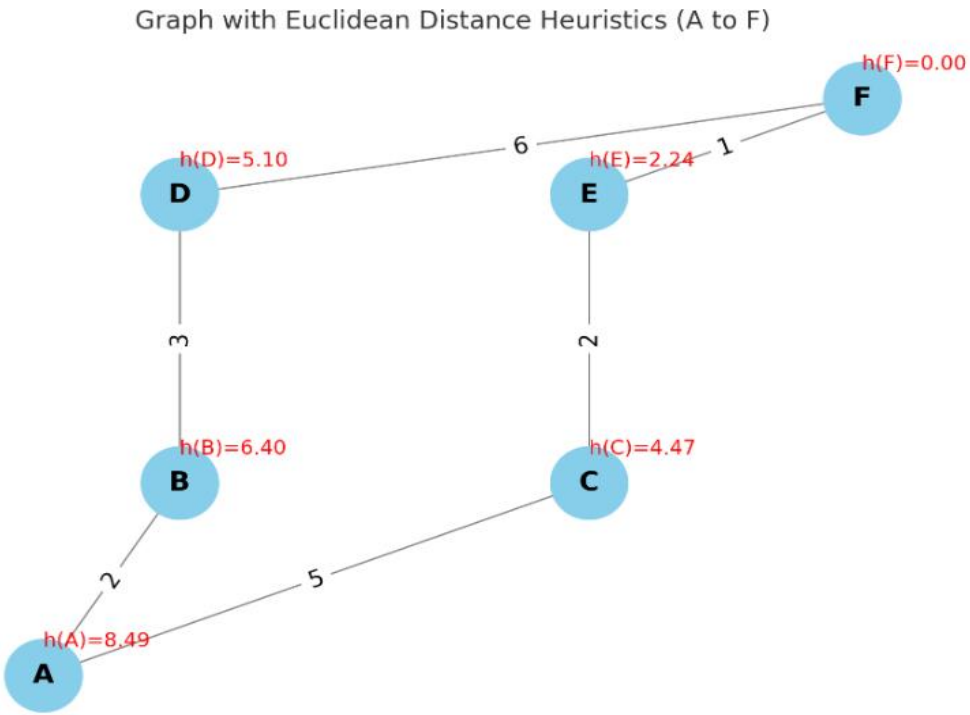
1. Start at Node A:
   - Heuristic value at A: $h(A)$=8.49
   - Neighbors: B and C

Compare the heuristic values of neighbors:

- $h(B)$=6.40
- $h(C)$=4.47

Choose C because it has a lower heuristic value than B.

Move to C.



Hill Climbing Algorithm

Step-by-Step Execution of Hill Climbing:

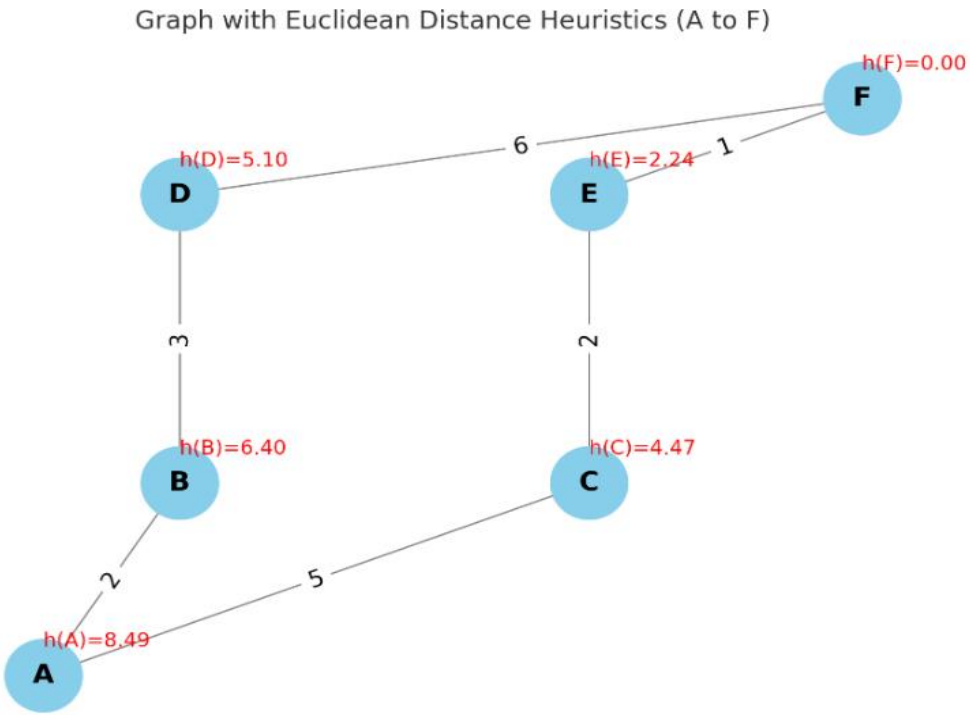
2. At Node C:
   - Heuristic value at C: $h(C)$=4.47
   - Neighbors: A (already visited) and E

Compare the heuristic values of neighbors:

- h(E)=2.2

Choose E, since it's the only valid neighbor and has a lower heuristic value.

Move to E.



Hill Climbing Algorithm

Step-by-Step Execution of Hill Climbing:

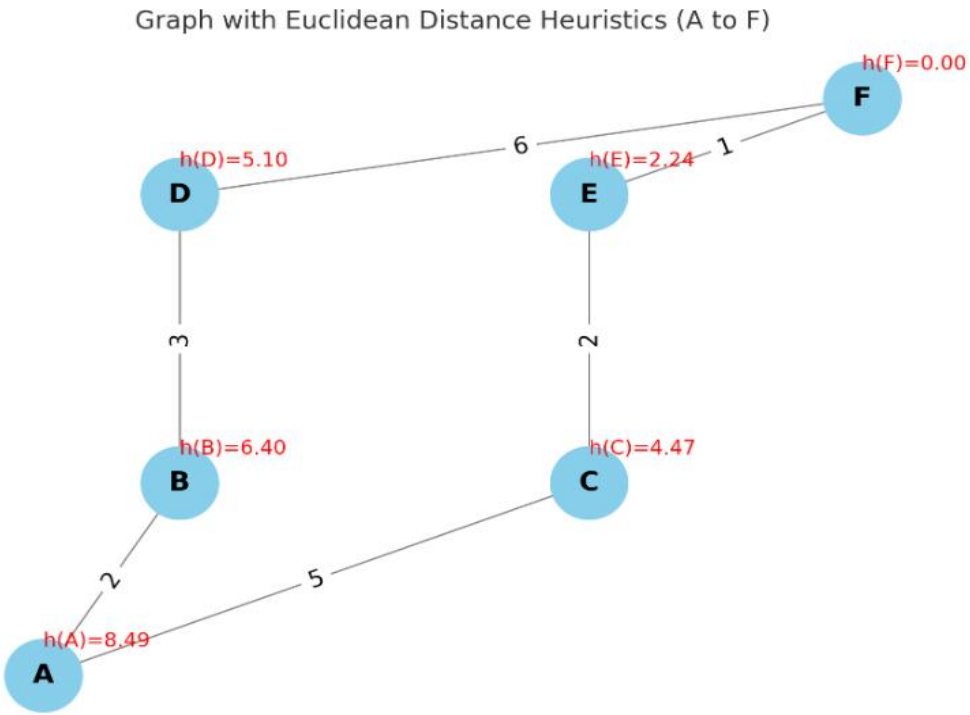
3. At Node E:
   - Heuristic value at E: h(E)=2.24
   - Neighbors: C (already visited) and F

Compare the heuristic values of neighbors:

- h(F)=0 (Goal)

Choose F, since it has the lowest possible heuristic value (goal node).

Move to F.

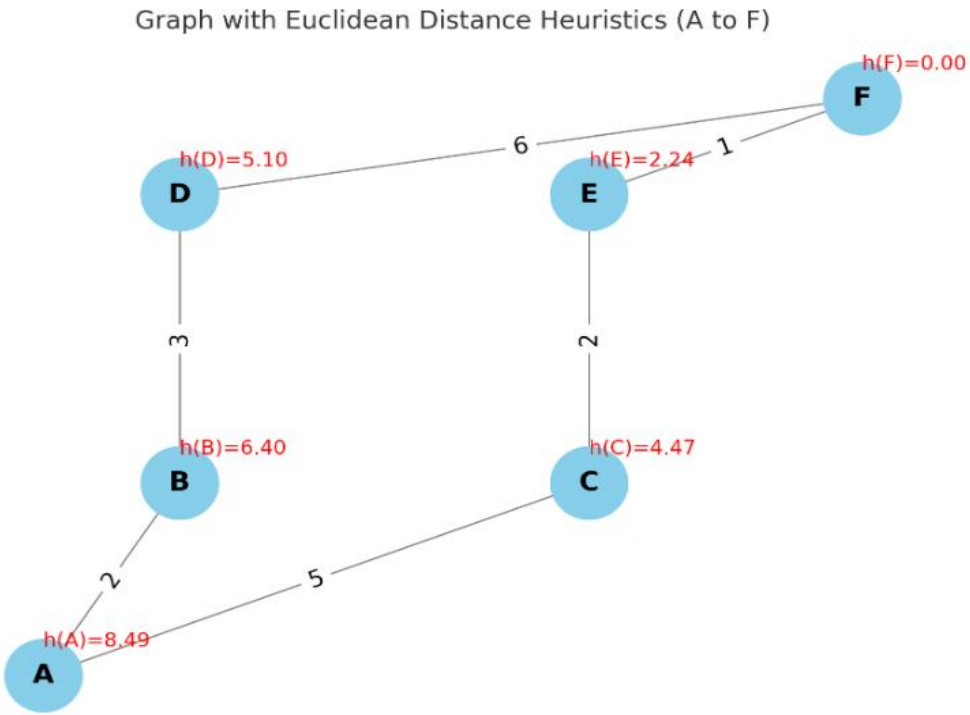


Hill Climbing Algorithm

Step-by-Step Execution of Hill Climbing:

4. At Node F (Goal):
   - The goal node has been reached

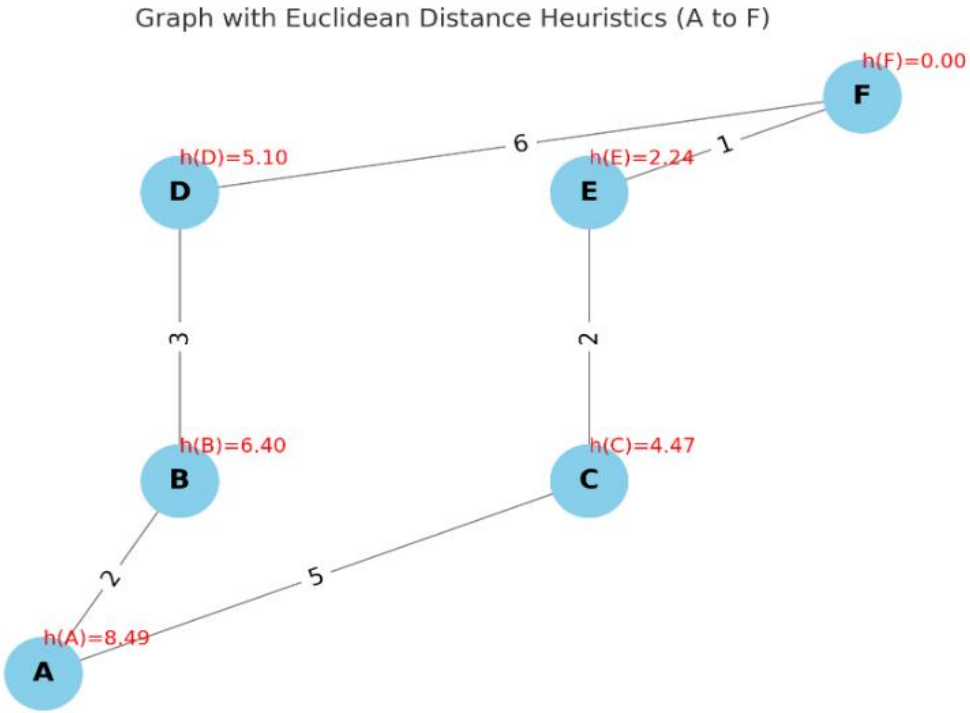
- Final Path Found by Hill Climbing:
  - The path is A → C → E → F.



Hill Climbing Algorithm

Why Did Hill Climbing Work Here?

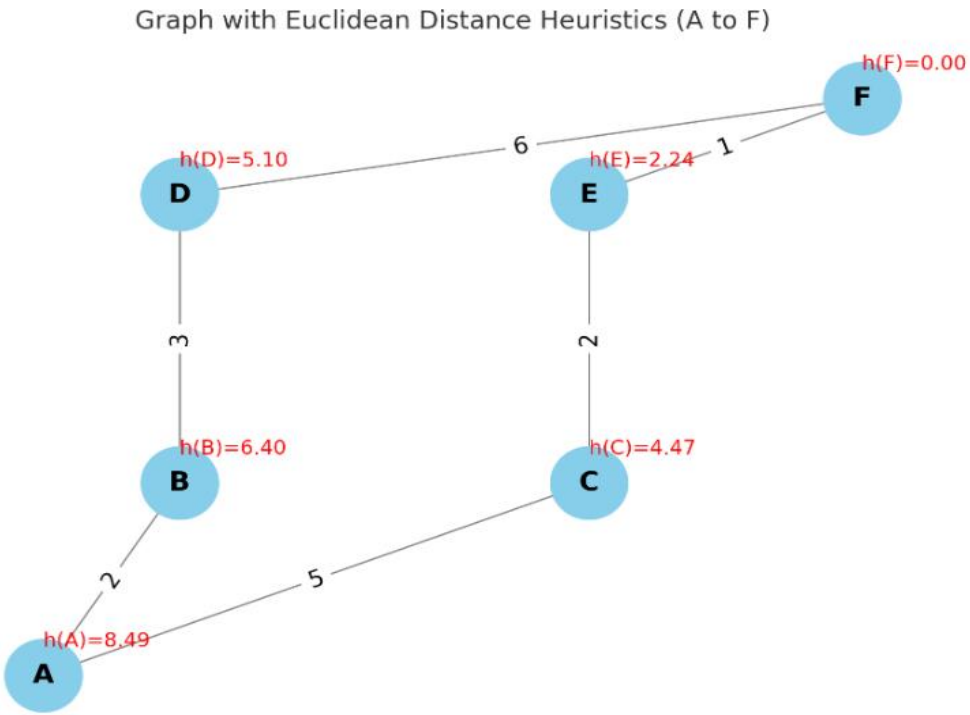
- Hill Climbing selected the neighbors with the lowest heuristic values at each step. It didn't backtrack or explore paths that seemed farther from the goal.
- It avoided B → D → F, even though it is a valid path, because C → E → F had a lower heuristic at each step.



Hill Climbing Algorithm

Limitations of Hill Climbing:

- **Hill Climbing** is a **greedy algorithm** and can get stuck in local optima. In this case, there were no local optima that prevented the algorithm from reaching the goal, but in more complex graphs, Hill Climbing might fail to find the global optimum.
- It doesn't backtrack or explore other potential paths once it makes a decision.



Hill Climbing Algorithm (Example 2)

Example on the Graph:

- Start at A.
  - Neighbors: B (cost = 1), C (cost = 4).
  - Move to B since it has the lower cost.
- Now at B:
  - Neighbors: A (already visited), D (cost = 3).
  - Move to D (next best option).
- Now at D:
  - Neighbors: B (already visited), F (cost = 6).
  - Move to F (goal reached).

Hill Climbing Path: A → B → D → F (Total cost = 1 + 3 + 6 = 10).

Hill Climbing Algorithm: Step-by-Step

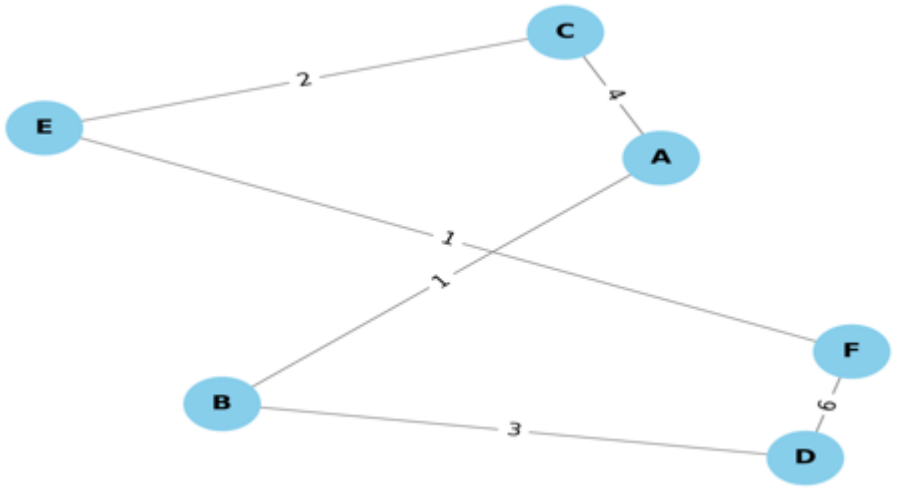
The goal is to reach node F from node A, minimizing the total cost, which is the sum of the edge weights along the path.

Graph Structure Recap:

Nodes: A, B, C, D, E, F

Edges and weights:

- A-B (cost = 1)
- A-C (cost = 4)
- B-D (cost = 3)
- C-E (cost = 2)
- D-F (cost = 6)
- E-F (cost = 1)



Objective: Reach F from A, minimizing the total cost.

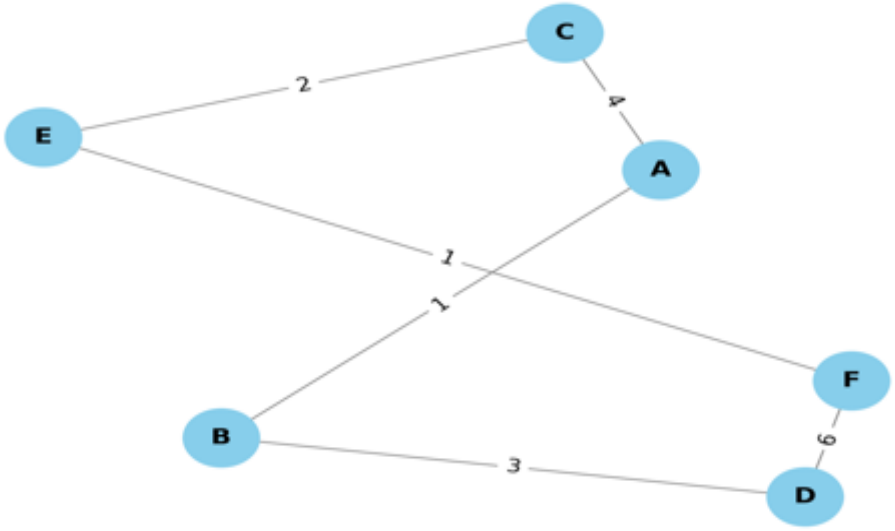
Hill Climbing Algorithm: Step-by-Step

Step 1: Start at Node A

- Neighbors of A: B (cost = 1), C (cost = 4)
- Decision: Since Hill Climbing is a greedy algorithm, it selects the node with the lowest cost from its neighbors. Here, node B has a lower cost than C.
- Move to B.

State After Step 1:

- Current node: B
- Path so far: A → B
- Total cost so far: 1



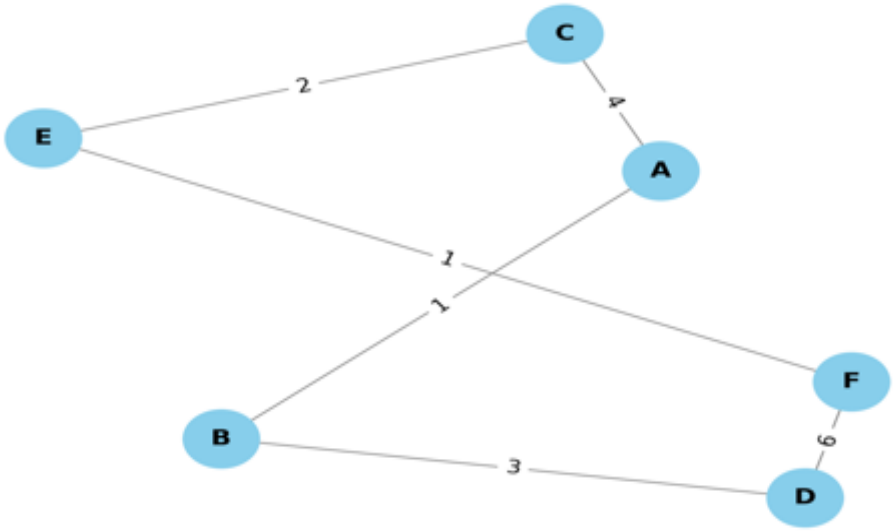
Hill Climbing Algorithm: Step-by-Step

Step 2: At Node B

- Neighbors of B: A (already visited), D (cost = 3)
- Decision: The only unvisited neighbor of B is D, so the algorithm moves to D.
- Move to D.

State After Step 2:

- Current node: D
- Path so far: A → B → D
- Total cost so far: 1 + 3 = 4



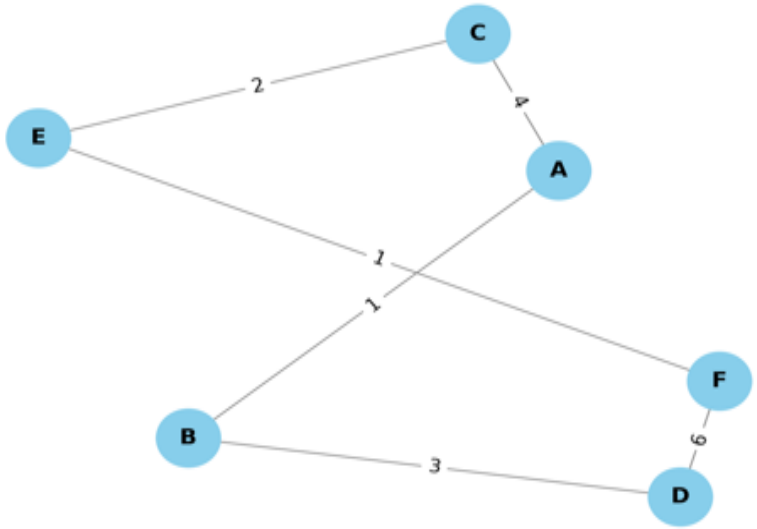
Hill Climbing Algorithm: Step-by-Step

Step 3: At Node D

- Neighbors of D: B (already visited), F (cost = 6)
- Decision: The only unvisited neighbor of D is F, so the algorithm moves to F.
- Move to F.

State After Step 3:

- Current node: F (goal reached)
- Path: A → B → D → F
- Total cost: 1 + 3 + 6 = 10



Hill Climbing Algorithm: Step-by-Step

Final Hill Climbing Path:

- Path: A → B → D → F
- Total cost: 10

Analysis of Hill Climbing:

- The algorithm found a path to the goal, but the total cost is 10.
- Hill Climbing greedily selected the lowest-cost neighbors at each step, but it didn't explore other parts of the graph that might have led to a lower-cost solution.
- In this case, the path through C and E offers a better solution, but Hill Climbing doesn't explore it because it only focuses on the current best option.

Self Study

- Simulated Annealing

Book Reference

Stuart Russell and Peter Norvig: “Artificial Intelligence a Modern Approach”, 3rd Edition, Pearson Education Asia. (Text Book) (Online) (Lectures) Chapter 3.